
Contextual Rollout Bandits for Reinforcement Learning with Verifiable Rewards

Xiaodong Lu^{1,2} Xiaohan Wang² Jiajun Chai² Guojun Yin² Wei Lin² Zhijun Chen¹ Yu Luo³
Fuzhen Zhuang⁴ Yikun Ban^{1†} Deqing Wang^{1*}

Abstract

Reinforcement Learning with Verifiable Rewards (RLVR) is an effective paradigm for improving the reasoning capabilities of large language models. However, existing RLVR methods utilize rollouts in an indiscriminate and short-horizon manner: responses of heterogeneous quality within each prompt are treated uniformly, and historical rollouts are discarded after a single use. This leads to noisy supervision, poor sample efficiency, and suboptimal policy updates. We address these issues by formulating rollout scheduling in RLVR as a contextual bandit problem and proposing a unified neural scheduling framework that adaptively selects high-value rollouts throughout training. Each rollout is treated as an arm whose reward is defined by the induced performance gain between consecutive optimization steps. The resulting scheduler supports both noise-aware intra-group selection and adaptive global reuse of historical rollouts within a single principled framework. We provide theoretical justification by deriving sublinear regret bounds and showing that enlarging the rollout buffer improves the achievable performance upper bound. Experiments on six mathematical reasoning benchmarks demonstrate consistent gains in performance and training efficiency across multiple RLVR optimization methods.

1. Introduction

From OpenAI O1 (Jaech et al., 2024) to DeepSeek R1 (DeepSeek-AI, 2025), recent advancements (Zou et al.,

If you have any questions, please contact via xiaodonglu@buaa.edu.cn or yikunb@buaa.edu.cn. †: Co-advised this work. ¹School of Computer Science and Engineering, Beihang University ²Meituan ³Huawei ⁴School of Artificial Intelligence, Beihang University. Correspondence to: Deqing Wang <dqwang@buaa.edu.cn>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

2025; Chen et al., 2025b) in large reasoning models have highlighted the crucial role of Reinforcement Learning with Verifiable Rewards (RLVR) in enhancing mathematical reasoning capabilities. By grounding policy optimization in verifiable reward signals, RLVR establishes a scalable self-improvement paradigm that iteratively corrects reasoning trajectories (i.e., rollouts) through trajectory generation and verification-driven learning.

Advanced RLVR methods (Zheng et al., 2025a; Yu et al., 2025a; Shao et al., 2024; Yang et al., 2026) typically employ group-relative policy optimization, which generates multiple responses for a single prompt to derive relative feedback signals. However, these methods often adopt an indiscriminate utilization of responses within a group, incorporating all generated samples into the gradient computation regardless of their individual learning value. In practice, due to the coarse granularity of rule-based rewards, responses within a group exhibit substantial quality heterogeneity, with many providing weak, noisy, or even misleading supervision, such as guessed-correct solutions (Yue et al., 2025; Arcuschin et al., 2025) and trajectories dominated by redundant reasoning steps (Li et al., 2025; Huang et al., 2025a). Failing to filter such low-utility rollouts not only incurs unnecessary computational overhead but can also distort policy updates, motivating the need for fine-grained, noise-aware selection within each rollout group.

Beyond intra-group noise, existing RLVR pipelines also suffer from a second fundamental limitation: *restricted data horizons*. Most methods rely exclusively on the latest batch of rollouts for optimization, discarding past trajectories after a single use. This myopic training paradigm leads to poor sample efficiency and limits attainable policy performance, as many high-value rollouts are never revisited. While recent efforts have explored reusing historical data (Hejna et al., 2024; Zhan et al., 2025), these approaches largely depend on static, hand-crafted heuristics and lack adaptivity to evolving training dynamics. How to systematically and adaptively reuse historical rollouts remains underexplored in RLVR.

To address the above two challenges, we formulate rollout scheduling as a *contextual bandit* problem, where each rollout is viewed as an arm, and the arm reward is derived from the policy performance gain between two consecutive opti-

mization steps induced by selecting that rollout. Building on this formulation, we propose a neural data scheduler plugin, named CBS, which decomposes naturally into two complementary contributions.

(i) *Intra-group rollout selection.* To address response-quality heterogeneity within groups, CBS introduces a contextual bandit-based scheduler that performs fine-grained selection within each rollout group. We design a compact 10-dimensional arm representation that encodes policy-optimization-related training dynamics (e.g., reward, advantage, entropy, and clipping statistics), and instantiate the scheduler as an MLP that predicts the future utility of each rollout under online feedback. This enables CBS to automatically prioritize informative responses while suppressing noisy or misleading ones during policy optimization.

(ii) *Global Adaptive Rollout Reuse.* To overcome the limited-horizon constraint of existing RLVR methods, CBS further extends the scheduler to a global selection regime over a replay buffer of historical rollouts. By unifying online and offline rollout selection under the same contextual bandit framework, CBS adaptively reuses high-value trajectories across training rounds, substantially improving sample efficiency and enabling long-horizon policy refinement.

We validate the effectiveness of CBS from both theoretical and empirical perspectives. Theoretically, under a simplified yet essential single-rollout selection setting, we establish a formal connection between RLVR data scheduling and contextual bandits, define the per-round regret, and derive a sublinear regret bound for the proposed neural scheduler. We also show that enlarging the replay buffer improves the achievable upper bound of policy performance, providing principled justification for global rollout reuse. Empirically, experiments on six widely used mathematical reasoning benchmarks demonstrate that CBS consistently improves both performance and training efficiency across three representative policy optimization methods. Ablation studies further verify the effectiveness of the proposed components.

Our contributions can be summarized as follows:

- **Unified Contextual-Rollout Bandit for RLVR.** We formulate rollout scheduling in RLVR as a contextual bandit problem and propose a unified neural scheduling framework that supports both (i) noise-aware intra-group rollout selection and (ii) adaptive global reuse of historical rollouts, enabling fine-grained supervision filtering and long-horizon data reuse within a single principled decision-making framework.
- **Theoretical Guarantees.** We establish a formal connection between RLVR data scheduling and contextual bandits, and derive sublinear regret bounds for the proposed scheduler under a simplified yet essential setting, providing theoretical justification for both local and

global rollout selection.

- **Empirical Effectiveness and Efficiency.** Extensive experiments on six mathematical reasoning benchmarks demonstrate that the proposed method consistently improves both performance and training efficiency across multiple RLVR optimization algorithms, with ablation studies validating the contribution of each component.

2. Related Work

2.1. RLVR for LLM Reasoning

Reinforcement Learning (RL) has emerged as a fundamental paradigm for the post-training of Large Language Models (LLMs), where the model is conceptualized as a policy π_θ that generates tokens as sequential actions to maximize cumulative reward signals. Initial developments in this field primarily focused on Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022; Dai et al., 2024; Christiano et al., 2017; Rafailov et al., 2023; Azar et al., 2024), which utilizes human preference data to either train reward models (Ouyang et al., 2022) or directly guide policy optimization (Rafailov et al., 2023). More recently, there has been a surge of interest in Reinforcement Learning with Verifiable Rewards (RLVR) for verifiable domains such as mathematics and code generation. These approaches (Zheng et al., 2025a; Yu et al., 2025a; Shao et al., 2024; Yang et al., 2026; 2025b) leverage rule-based rewards—such as deterministic correctness or execution feedback—to incentivize the model’s reasoning capabilities and have demonstrated significant potential in improving complex problem-solving tasks. Despite these advancements, contemporary RLVR methods typically utilize generated trajectories in an indiscriminate and local manner, ignoring the heterogeneous learning utility of different rollouts.

2.2. Data Selection for RLVR

Data selection is a critical technique for effective LLM post-training, which has been extensively studied in SFT (Zhou et al., 2023; Chen et al., 2023; Lu et al., 2024; Muennighoff et al., 2025; Ivison et al., 2025; Xia et al., 2024) and RLHF (Das et al., 2025; Mahmud et al., 2025; Mehta et al., 2023; Shen et al., 2024; Huang et al., 2025b) scenarios to mitigate data noise and enhance training efficiency. With the rising success of RLVR, recent works have begun to explore data selection mechanisms within this paradigm (Zhang et al., 2025; Zheng et al., 2025b; Hejna et al., 2024; Zhan et al., 2025; Dou et al., 2025; Xu et al., 2025). For instance, GRESO (Zheng et al., 2025b) leverages historical success-failure records of prompts to identify and filter out zero-advantage samples that are unlikely to contribute to policy improvement; while ExGRPO (Zhan et al., 2025) enhances training efficiency by augmenting on-policy data with historical rollouts, specifically prioritizing high-entropy responses

generated under prompts of intermediate difficulty. However, current data selection methods for RLVR predominantly rely on manual and method-specific heuristics that are labor-intensive and lack flexibility, leaving the potential for more adaptive and automated mechanisms largely unexplored.

3. Problem Formulation

3.1. Preliminary

3.1.1. GROUP RELATIVE POLICY OPTIMIZATION

Group relative policy optimization methods are widely adopted in existing RLVR paradigm. Specifically, given a question-answer pair (q, a) , the group relative policy optimization method first generates a group of responses $\{o_i\}_{i=1}^G$ using the current policy. The advantage of each response is then estimated by $A_i = \frac{v_i - \text{mean}(\{v_i\}_{i=1}^G)}{\text{std}(\{v_i\}_{i=1}^G)}$, where v_i is the reward of the i -th response. The policy π_θ is optimized by maximizing

$$\mathcal{J}_\theta = \mathbb{E}_{(q,a) \sim D, \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} A_i \Phi(A_i, r_{i,t}(\theta), \epsilon_{\text{low}}, \epsilon_{\text{high}}) \right], \quad (1)$$

where $\mathcal{D}$ is the training dataset, Φ is a method-specific clip function, $r_{i,t}(\theta) = \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\text{old}}(o_{i,t}|q, o_{i,<t})}$ is the importance sampling weight at the t -th token of o_i , $\pi_{\text{old}}(\cdot)$ indicates the policy to generate responses $\{o_i\}_{i=1}^G$, ϵ_{low} and ϵ_{high} are the hyperparameters controlling the clip lower bound and the clip upper bound respectively. In this paper, we consider three popular policy optimization methods: GRPO (Shao et al., 2024), DAPO (Yu et al., 2025b), and GSPO (Zheng et al., 2025a), and refer to Appendix A for their detailed introduction.

3.1.2. RLVR TRAINING PIPELINE

The RLVR training can be considered as a multi-round policy optimization problem. Assume the standard RLVR training spans T rounds. In each round t , a batch of question-answer pairs, $\mathcal{B}_t = \{(q_1^t, a_1^t), \dots, (q_B^t, a_B^t)\}$, is first randomly sampled from the training dataset D , and then a group of responses $\{o_{i,j}^t\}_{j \in [G]}$ are generated by the policy model $\pi_{\theta_{t-1}}$ for each question q_i , forming a batch of rollouts $\mathcal{R}_t = \{x_{i,j}^t\}_{i \in [B], j \in [G]}$, where $x_{i,j}^t = (q_i^t, a_i^t, o_{i,j}^t)$ ¹ indicates a piece of rollout data. Then the policy parameter

¹We only show partial fields in $x_{i,j}^t$; any quantity with the same index is an attribute of that rollout (i.e., $v_{i,j}^t$ indicates reward).

is updated by²

$$\theta_t = \theta_{t-1} + \eta_1 \nabla_{\theta_{t-1}} \mathcal{J}_\theta(\mathcal{R}_t), \quad (2)$$

where $\mathcal{J}_\theta(\mathcal{R}_t)$ indicates the target function in Equation (1), computed using $\mathcal{R}_t$. A visual illustration for this pipeline can be found in Appendix A.2.

3.2. Contextual Rollout Bandit

We formulate data scheduling as a contextual bandit problem (Ban et al., 2022). Specifically, we introduce a scheduler that, at each round t , selects a high-quality subset $\bar{\mathcal{C}}_t$ with $|\bar{\mathcal{C}}_t| = K$ from a given rollout set $\mathcal{C}_t$. Let $f(\cdot)$ denote a rollout encoding function that maps a subset into a representation. Then, each candidate subset $\mathcal{C}' \subseteq \mathcal{C}_t$ of size K can be viewed as an arm with representation $H' = f(\mathcal{C}')$. Depending on the candidate arm space and the data source, we study the following two scheduling problems.

Definition 3.1 (Intra-Group Scheduling Problem).

We aim to *downsample training rollouts* by retaining only high-quality samples within each group. Formally, for each group of rollouts $\{x_{i,j}^t\}_{j=1}^G \subset \mathcal{R}_t$, we select a top- $p\%$ subset indexed by $\mathcal{S}_i^t \subseteq \{1, \dots, G\}$ with $|\mathcal{S}_i^t| = \lfloor p\% \cdot G \rfloor$, yielding the selected set $\{x_{i,j}^t\}_{j \in \mathcal{S}_i^t}$. Denoting the selected rollouts as $\bar{\mathcal{C}}_t = \bigcup_{i=1}^B \{x_{i,j}^t\}_{j \in \mathcal{S}_i^t}$, the policy is optimized by training on $\bar{\mathcal{C}}_t$ in Equation (2).

Definition 3.2 (Global Scheduling Problem).

We aim to *reuse high-quality historical rollouts*. Specifically, we maintain a FIFO buffer that stores rollouts from the most recent L iterations, $\mathcal{C}_t = \bigcup_{t'=t-L+1}^t \mathcal{R}_{t'}$. At round t , we select K high-quality rollouts from $\mathcal{C}_t$ to construct a training set $\bar{\mathcal{C}}_t \subseteq \mathcal{C}_t$, where $|\bar{\mathcal{C}}_t| = K$. We set $K = |\mathcal{R}_t|$ to match the amount of the latest rollouts. The policy is then optimized by training on $\bar{\mathcal{C}}_t$ in Equation (2).

The two scheduling problems will both generate a rollout sequence $\bar{\mathcal{C}}_1, \bar{\mathcal{C}}_2, \dots, \bar{\mathcal{C}}_T$ and corresponding parameter sequence $\theta_1, \dots, \theta_T$. Assume we have an unknown reward function $R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t)$ that evaluates the performance change from $\pi_{\theta_{t-1}}$ to π_{θ_t} , after training on $\bar{\mathcal{C}}_t$. Then the target of the scheduling sequence is to maximize the cumulative reward

$$R_T = \sum_{t=1}^T R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t). \quad (3)$$

Due to the high storage and computational cost for the LLM policy, it is impractical to accurately evaluate the perfor-

²We adopt the SGD optimizer here for simplicity.

mance change $\theta_{t-1} \rightarrow \theta_t$. Therefore, we resort to a computationally efficient surrogate that captures the true performance gain, and define the reward function as the metric gain between two consecutive rollout data $\mathcal{R}_t, \mathcal{R}_{t+1}$.³

Definition 3.3 (Performance Gain Reward). Let $V_t = \text{mean}(\{v_i^t | x_i^t \in \mathcal{R}_t\})$ and $E_t = \text{mean}(\{e_i^t | x_i^t \in \mathcal{R}_t\})$ denote the average reward and entropy for the t step rollout data, with e_i^t being defined as

$$e_i^t = \frac{-1}{|o_i^t|} \sum_{k=1}^{|o_i^t|} \sum_{v \in \mathcal{V}} \pi_{\theta_{t-1}}(v | q_i^t, o_{i,<k}^t) \log \pi_{\theta_{t-1}}(v | q_i^t, o_{i,<k}^t), \quad (4)$$

where $\mathcal{V}$ is the vocabulary of the LLM policy. Then the reward function is defined as

$$R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) = V_{t+1} - V_t - w_e \mathbb{I}[E_t > e_{\min}] * (E_{t+1} - E_t), \quad (5)$$

where $w_e, e_{\min}$ are hyperparameters and $\mathbb{I}[\cdot]$ is an indicator function.

Since each mini-batch $\mathcal{B}_t$ is randomly sampled from the training set, V_t provides a stochastic estimate of the expected performance of θ_{t-1} on the training set $\mathcal{D}$, and thus $V_{t+1} - V_t$ reflects the performance improvement between consecutive updates. The term $w_e \mathbb{I}[E_t > e_{\min}](E_{t+1} - E_t)$ serves as an entropy control term, penalizing entropy increases once the entropy exceeds a predefined threshold $e_{\min}$, preventing training collapse (shown in Section 6.3). It is also worth noting that the deviation between the stochastic estimate and its corresponding expectation is bounded (Lemma B.2).

4. Methodology

As shown in Figure 1, we present CBS, a general data selection plugin that can be seamlessly integrated into existing RLVR methods for adaptive rollout selection. CBS employs a neural reward estimator $s_\phi(\cdot)$ that predicts arm-level rewards for rollout selection, and continually adapts its parameters based on online feedback. In the following, we first introduce the overall framework, then describe the detailed configuration, and finally explain how to employ the scheduler to address the two scheduling problems. The overall workflow is summarized in Algorithm 1.

³From here on, we flatten the two-dimensional indexing into a single dimension, using x_i^t to denote the i -th data instance in $\mathcal{R}_t$.

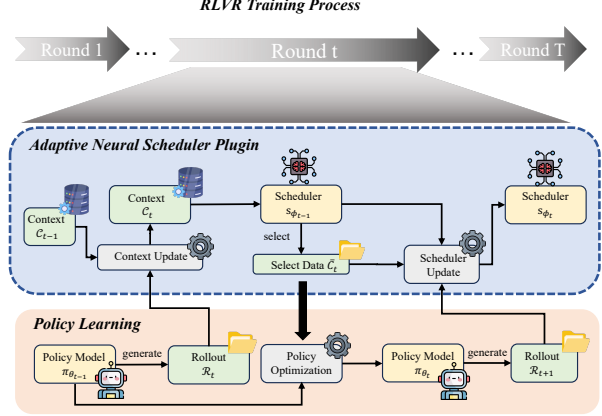


Figure 1. Workflow of CBS, introducing a neural scheduler plugin that augments RLVR training by selectively using rollouts. The scheduler also updates based on feedback.

4.1. Overall Framework of CBS

Subset selection via sample scoring. Scoring each rollout subset $\mathcal{C}' \subset \mathcal{C}_t$ is infeasible due to the combinatorial explosion in the number of arms. We therefore decompose subset-level scoring into per-rollout evaluation. Specifically, let $s_{\phi_{t-1}}$ denote the scheduler at round t , for each rollout $x_i^t \in \mathcal{C}_t$, we construct individual representations $h_i^t = f(x_i^t)$ and score it via $s_{\phi_{t-1}}(h_i^t)$. Selecting the optimal size- K rollout subset $\bar{\mathcal{C}}_t$ then reduces to selecting the top- K rollouts. Notably, this decomposition can be justified by a first-order Taylor expansion of the group-level reward, which yields a linear approximation where the overall reward is expressed as the sum of per-sample rewards. Similar linearizations of subset utility are also seen in prior work on data selection for SFT and RLHF, as they enable more efficient subset selection (Xia et al., 2024; Saunshi et al., 2023). Detailed discussion can be found in Appendix C.

Sample reward. To provide feedback for the sample scoring, we correspondingly dispatch the group-level reward $R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t)$ to sample-level rewards $R(\bar{x}_i^t, \theta_{t-1}, \theta_t)$ for each $\bar{x}_i^t \in \bar{\mathcal{C}}_t$. Inspired by GRPO-style objectives in Equation (1), where each importance weight is scaled by an advantage term, we adopt a simple advantage-based assignment of:

$$R(\bar{x}_i^t, \theta_{t-1}, \theta_t) = \frac{|\bar{A}_i^t|}{\sum_{\bar{x}_j^t \in \bar{\mathcal{C}}_t} |\bar{A}_j^t|} R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t), \quad (6)$$

where the contribution of each sample is related to its advantage magnitude.

Scheduler update. We further update the scheduler parameters online, allowing it to adapt its scheduling policy in response to reward feedback. Specifically, after receiving the sample feedback, we will optimize the scheduler

according to the sample reward, which is

$$\phi_t = \phi_{t-1} - \eta_2 \nabla_{\phi_{t-1}} \frac{1}{K} \sum_{i=1}^K \left(s_{\phi_{t-1}}(\bar{h}_i^t) - R(\bar{x}_i^t, \theta_{t-1}, \theta_t) \right)^2 \quad (7)$$

where η_2 is the learning rate of the scheduler. Since the feedback at round t depends on the rollout in round $t+1$, we perform the scheduler update just after the rollout phase at round $t+1$ (see Algorithm 1), before scheduling rollouts.

4.2. Detailed Configuration

Arm representation. A notable amount of prior work has shown that training dynamics associated with rollout data—such as advantage, entropy, and clip ratio—are strongly correlated with the underlying data value (Zhan et al., 2025; Liu et al., 2025b; Zhu et al., 2025). Inspired by this, we encode each arm x_i^t into a 10-dimensional training-dynamic vector $h_i^t = f(x_i^t) \in \mathbb{R}^{10}$. As shown in Table 1, these features can be divided into four groups of *reward-related signals*, *length-related signals*, *policy perception metrics*, and *data interaction history*. Although each feature is individually simple, their proper combination yields a rich representation of rollouts, making it possible to identify informative patterns, such as high or low entropy under fresh or stale data regimes. Besides, the *policy perception metrics* and *data interaction history* in Table 1 dynamically evolve whenever a rollout is used or a new round begins, enabling a reused rollout to be presented as different representations. Notably, constructing these representations incurs negligible computational overhead (will be shown in Section 6.2): most features are off-the-shelf metrics readily available in standard RL frameworks (e.g., verl), and the relatively complex policy perception metrics can be efficiently obtained during each forward pass without requiring an additional forward computation.

Scheduler architecture and decision. We design the scheduler as a multi-layer perceptron (MLP). For scheduling, we select the top- K rollouts based on their scores. Besides, to balance exploration and exploitation, we use epsilon-greedy to select between the highest-scoring and the newest data. Specifically, the i -th data $\bar{x}_i^t \in \bar{\mathcal{C}}_t$ is selected by

$$\bar{x}_i^t = \begin{cases} \arg \max_{x \in \mathcal{C}_t \setminus \{\bar{x}_j^t\}_{j=1}^{i-1}} s_{\phi_{t-1}}(h) & \text{w.p. } 1 - \epsilon_t, \\ \arg \min_{x \in \mathcal{C}_t \setminus \{\bar{x}_j^t\}_{j=1}^{i-1}} f_d(x) & \text{w.p. } \epsilon_t. \end{cases} \quad (8)$$

where h indicates the representation of x and $f_d(x) \in \mathbb{R}$ is the *sample age* of x in Table 1. The probability changes as

$$\epsilon_t = \begin{cases} 1.0 & t \leq T_w, \\ \max(\epsilon_0 - (t-1) * \Delta_\epsilon, \epsilon_{\min}) & t > T_w, \end{cases} \quad (9)$$

Table 1. The 10-dimensional training-dynamics representation is defined for each rollout $x_i^t \in \mathcal{C}_t$.

Description	Mathematical Formulation
Reward-related signals (<i>Intra-group relative merit</i>)	
Reward	v_i^t
Advantage	A_i^t
Mean group reward	$\text{mean}(\{v_j^t \mid x_j^t \text{ is in the same group as } x_i^t\})$
Std of group reward	$\text{std}(\{v_j^t \mid x_j^t \text{ is in the same group as } x_i^t\})$
Length-related signals (<i>Response volume and integrity</i>)	
Normalized length ¹	$ o_i^t / L_{\max}$
Truncation flag	$\mathbb{I}[o_i^t \text{ is truncated}]$
Policy perception metrics (<i>Model state projected onto data</i>)	
Entropy ²	$-\frac{1}{ o_i^t } \sum_{j=1}^{ o_i^t } \sum_{v \in \mathcal{V}} \pi_{\theta_{t-}}(v q_i^t, o_{i,<j}^t) \cdot \log \pi_{\theta_{t-}}(v q_i^t, o_{i,<j}^t)$
Clip ratio	$\frac{1}{ o_i^t } \sum_{j=1}^{ o_i^t } \mathbb{I}[r_{i,j}^t(\theta_{t-}) \text{ is clipped}]$
Data interaction history (<i>Usage and freshness</i>)	
Usage count ³	$\sum_{k=1}^{t-1} \mathbb{I}[x_i^t \in \bar{\mathcal{C}}_k]$
Sample age ⁴	$t - \max\{k \mid k < t, o_i^t \in \mathcal{R}_k\}$

¹ $L_{\max}$ denotes the maximum response length.

² t^- denotes the latest round that x_i^t is used for policy optimization, or the round at which it was generated if it has never been used. The same definition applies to the clip ratio.

³ The number of times x_i^t is used for policy optimization.

⁴ The number of rounds elapsed since the sample was generated.

where $T_w, \Delta_\epsilon, \epsilon_{\min}, \epsilon_0$ are hyperparameters controlling the warmup step, probability decay weight, minimum explore probability, and initial explore probability, respectively.

Reward augmentation. To improve the training stability of the scheduler, we normalize the group reward by Exponential Moving Average (EMA). Specifically, we first replace the raw reward gain in (5) with an EMA one and squeeze it within $[0, 1]$ by Sigmoid, which is

$$\hat{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) = \left[\text{Sigmoid} \left(\frac{(V_{t+1} - V_t) - \mu_t}{\sqrt{\sigma_t}} \right) - w_e \mathbb{I}[E_t > e_{\min}](E_{t+1} - E_t) \right], \quad (10)$$

$$\mu_t = (1 - \alpha)\mu_{t-1} + \alpha(V_{t+1} - V_t),$$

$$\sigma_t = (1 - \alpha)\sigma_{t-1} + \alpha[(V_{t+1} - V_t) - \mu_t]^2$$

where α is a hyperparameter to control the degree of smoothness. Additionally, we find that the denominator in (6) yields very small sample weights when many rollouts are selected, leading to unstable scheduler training. We therefore remove the denominator and redefine the sample reward as

$$\hat{R}(\bar{x}_i^t, \theta_{t-1}, \theta_t) = |A_i^t| * \hat{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) \quad (11)$$

Finally, we summarize the overall workflow in Algorithm 1.

4.3. Employing the Scheduler for the Two Scheduling Problems

Intra-group scheduling. For intra-group scheduling, we set the context set to the latest batch of rollouts (i.e., $\mathcal{C}_t = \mathcal{R}_t$) and organize rollouts by group. In each scheduling round, the scheduler respectively selects the top- $p\%$ rollouts within each group and aggregates them to form the selected set $\bar{\mathcal{C}}_t$.

Global scheduling problem. For the global scheduling problem, we employ a FIFO replay buffer to store rollouts from the latest L steps and perform global selection to choose K rollouts to form the set $\bar{\mathcal{C}}_t$. Moreover, since the *policy perception metrics* and *data interaction history* in Table 1 evolve across training rounds—where *sample age* increases over time, while other features change only when a rollout is selected for policy optimization—we update these features at the end of each round (see Algorithm 1).

5. Theoretical Analysis

In this section, we formally describe the connection between the data scheduling and contextual bandit problem to provide intuitive motivation for CBS. For clarity, we consider the single-sample selection setting, in which we select one rollout per round, and the sample reward coincides with the subset reward. We refer to Appendix C for the analysis of batch selection. For reward function, according to Lemma B.2, the raw scheduler reward satisfies $\mathcal{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) \in [-A, A]$, where $A = 1 + w_e \log N$ and $N = |\mathcal{V}|$ is the vocabulary size. Therefore, in the analysis, we use the normalized reward

$$\bar{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) = \frac{\mathcal{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) + A}{2A}.$$

Then $\bar{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) \in [0, 1]$, which satisfies the bounded label condition required by the neural regression lemmas. Notably, this affine normalization only scales each regret gap by the constant factor $1/(2A)$, and thus does not change the order of the regret bound. All proofs for this section can be found in Appendix B.

Under the above setting, Algorithm 1 naturally reduces to a standard contextual bandit problem. Let $M = |\mathcal{C}_t|$ denote the context size. At each round t , a set of arms $\mathcal{C}_t = \{x_1^t, \dots, x_M^t\}$ and their associated representations $H_t = \{h_1^t, \dots, h_M^t\}$ are presented to the scheduler. An arm $a_t \in [M]$ is selected, and a reward r_t is observed for the chosen arm. Let $V_t^s = \bar{R}(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t)$ be the random variable denoting the scheduler reward at round t . Rewriting the policy optimization and the expected rewards as functions below,

$$\begin{aligned} \mathcal{A}(x, \theta_{t-1}) &= \theta_{t-1} + \eta \nabla_{\theta_{t-1}} J_\theta(x), \\ g(\theta_t) &= \mathbb{E}_{\mathcal{B}_{t+1} \sim \mathcal{D}, \mathcal{R}_{t+1} \sim \pi_{\theta_t}(\cdot | \mathcal{B}_t)} [V_t^s], \end{aligned} \quad (12)$$

the r_t can be considered as a stochastic estimate of function $g(\mathcal{A}(x_{a_t}^t, \theta_{t-1}))$. Therefore, we can define the optimal arm as $a_t^* = \arg \max_{x \in \mathcal{C}_t} g(\mathcal{A}(x, \theta_{t-1}))$. The cumulative regret is then given as

$$R_T = \sum_{t=1}^T \left[g(\mathcal{A}(x_{a_t^*}^t, \theta_{t-1})) - g(\mathcal{A}(x_{a_t}^t, \theta_{t-1})) \right]. \quad (13)$$

In the following, we interchangeably use $g(\mathcal{A}(x_a^t, \theta_{t-1}))$ and r_a^t for notation simplicity.

Our theoretical results build on the Neural Tangent Kernel (NTK) framework, which provides a tractable characterization of training dynamics in the overparameterized regime. We instantiate the selector network $s_\phi(\cdot)$ as a fully connected network with depth $L \geq 2$ and width m (details in Appendix B.1). We slightly abuse the notations and rewrite the arm representations $\{h_i^t\}_{i \in [M], t \in [T]}$ and rewards $\{r_i^t\}_{t \in [T], i \in [M]}$ into $\{h_i\}_{i \in [TM]}$ and $\{r_i\}_{i \in [TM]}$ respectively. Let $H \in \mathbb{R}^{TM \times TM}$ denotes the NTK matrix of $\{h_i\}_{i \in [TM]}$ (defined in Appendix B.1). We adopt the following two standard assumptions for contextual bandit analysis (Zhou et al., 2020; Zhang et al., 2021; Dai et al., 2023; Jia et al., 2022; Ban et al., 2024).

Assumption 5.1. For any $i \in [K]$, $t \in [T]$, $\theta \in \mathbb{R}^d$, $\|h_i^t\|_2 = 1$ and $g(\theta) \in [0, 1]$.

Assumption 5.2. There exists $\lambda_0 > 0$ that $H \succeq \lambda_0 I$.

The assumptions are mild: the first is satisfied when the RLVR reward is bounded in $[0, 1]$ and the arm representations are ℓ_2 -normalized before being fed into s_ϕ , while the second holds as long as there are no identical representations among $\{h_i\}_{i \in [TM]}$.

We first investigate how the buffer size of global scheduling impacts the final policy performance. Holding all other hyperparameters fixed, we define $\varphi(M) = \max_\phi \mathbb{E}[\max_{t \in [T]} g(\theta_t)]$ as the optimal expected performance achievable during the training process, by setting buffer size to M and selecting the scheduler policy s_ϕ . The expectation is taken with respect to the randomness in the training procedure (i.e., rollout phase). We then state the following theorem.

Theorem 5.3. Let B denote the batch size and G denote the group size. There exist constants $C_1, C_2 > 0$ such that, if $m \geq C_1$ and $L \geq C_2$, then for any $i, j \in [T]$ with $i \geq j$, it holds that

$$\varphi(iBG) \geq \varphi(jBG).$$

Theorem 5.3 emphasizes the benefits of reusing historical rollouts, demonstrating that selectively leveraging rollouts from multiple prior rounds—not just the most recent

round—can enhance the performance upper bound of the policy model. Next, we present the regret bound for CBS.

Theorem 5.4. *Let $r = [r_1, \dots, r_{TM}] \in \mathbb{R}^{TM}$ be the stacked reward vector. If Assumption 5.1 and Assumption 5.2 hold, for any $\delta \in (0, 1)$, there exists a choice of scheduler hyperparameters (m, η_2) and a decay schedule for the exploration probability ϵ_t such that, with probability at least $1 - \delta$ over scheduler initialization,*

$$\mathbb{E}[R_T] \leq \mathbb{E} \left[\tilde{O} \left(\sqrt{T} \left(\tilde{d} + S \sqrt{\tilde{d}} \right) + \sqrt{T} \right) \right],$$

where $S = \sqrt{r^T H^{-1} r}$ and $\tilde{d} = \frac{\log \det(I+H)}{\log(1+TM)}$.

Theorem 5.4 provides a more fine-grained characterization of CBS. First, it shows that the regret increases sublinearly with the time horizon T , achieving a data-dependent $\tilde{O}(\sqrt{T})$ regret bound. Second, it reveals that the regret is intrinsically related to the effective dimension $\tilde{d}$ of the neural tangent kernel (NTK) and the geometric structure of the reward captured by S , where simpler feature structures (small $\tilde{d}$) and better-aligned rewards (small S) will lead to a smaller regret. Notably, in favorable structured regimes, $\tilde{d}$ can remain controlled. For example, (Zhou et al., 2020) (Remark 4.4) and (Zhang et al., 2021) (Remark 3.3) show that when the NTK matrix of arm representations has an underlying low-dimensional structure, $\tilde{d}$ can remain bounded.

6. Experimental Results

6.1. Experimental Settings

Models and datasets. We conduct experiments using Qwen3 Base models (Yang et al., 2025a) of various sizes, namely Qwen3-1.7B-Base, Qwen3-4B-Base, and Qwen3-8B-Base. Following (Yang et al., 2025b), the training set is constructed by combining the DAPO dataset (Yu et al., 2025a) and a subset of the Math dataset (Hendrycks et al., 2021) containing level 3-5 problems, yielding a total of 21723 mathematical problems. For evaluation, we adopt a suite of benchmark datasets, including AIME24, AIME25, AMC23, MATH500 (Hendrycks et al., 2021), Minerva (Lewkowycz et al., 2022), and Olympiad (He et al., 2024).

Implementation details. We implement CBS based on verl (Sheng et al., 2025). Maximum response length is set to 4096 for all experiments. Following (Zhan et al., 2025; Yang et al., 2025b), we report Avg@4 for datasets with larger problems (i.e., MATH500, Minerva, Olympiad) and Avg@32 for datasets with smaller problems (i.e., AIME24, AIME25 and AMC23). More details regarding the experimental setup are provided in the Appendix D. The code is available at https://github.com/lxd99/CBS_public.

6.2. Main Results

Intra-group scheduling improves both performance and computational efficiency. As shown in Table 2, integrating CBS* into representative policy optimization methods consistently yields substantial performance gains across different model sizes, demonstrating the effectiveness and robustness of our adaptive selection strategy. In particular, CBS* achieves average relative improvements of 5.0%, 5.8%, and 2.0% for GRPO, DAPO, and GSPO, respectively. Moreover, Figure 2(b) shows that CBS* reduces the average policy optimization time by 50.4% across the three methods, as fewer training trajectories are required per actor update. These results indicate that prioritizing a subset of high-value rollouts provides a more effective and efficient paradigm for RLVR training compared to using the full rollout set.

Global scheduling further improves training efficiency.

As shown in Table 2, CBS also consistently enhances different policy optimization methods, yielding relative improvements of 4.0%, 8.5%, and 1.7% for GRPO, DAPO, and GSPO, respectively. Both CBS and CBS* continue to improve performance even when a rule-based data filter is applied (e.g., DAPO with dynamic sampling), suggesting that heuristic selection rules are insufficient to capture the complex and dynamically evolving utility of rollout data during training. Furthermore, Figure 2(a) shows that CBS achieves higher training rewards than both CBS* and the original policy optimization baseline, indicating that appropriately reusing historical rollouts can further improve training efficiency. Importantly, the computational overhead of the scheduler is negligible: as shown in Table 3, buffer-related operations (e.g., scheduler updates and arm representation construction) account for less than 4% of total training time and decrease further as model size increases. Due to space limitations, more training dynamics curves are provided in Appendix E.1.

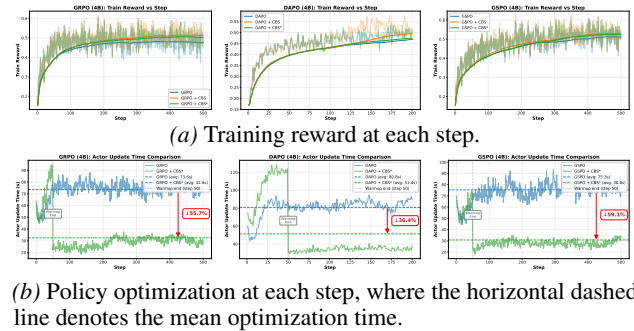


Figure 2. Train dynamics of different RLVR methods.

6.3. Ablation Study

To validate the effectiveness of the proposed submodules, we conduct an ablation study by comparing CBS with the

Table 2. Performance of different RLVR methods when equipped with CBS, where CBS indicates the global scheduler and the CBS* indicates the intra-group scheduler. Relative improvement (Rel. Impr.) is computed w.r.t. the corresponding policy optimization method, based on the *Average* score.

Method	AIME24	AIME25	AMC23	MATH500	Minerva	Olympiad	Average	Rel. Impr.
<i>Qwen3-1.7B-Base</i>								
Base Model	1.77	0.73	15.31	29.10	9.83	11.09	11.31	–
GRPO	9.58	4.38	42.97	60.95	26.29	29.30	28.91	–
GRPO + CBS	8.12	3.12	45.78	63.90	28.95	31.16	30.17	+4.4%
GRPO + CBS*	11.98	7.81	45.47	64.80	26.10	30.30	31.08	+7.5%
DAPO	6.35	4.69	37.97	58.70	20.59	27.23	25.92	–
DAPO + CBS	8.96	4.90	46.48	63.65	26.01	29.97	30.00	+15.7%
DAPO + CBS*	10.52	6.04	45.16	62.45	24.82	31.01	30.00	+15.7%
GSPO	10.73	5.52	43.52	65.95	27.94	31.53	30.87	–
GSPO + CBS	11.35	5.83	46.72	65.70	28.22	33.09	31.82	+3.1%
GSPO + CBS*	12.50	7.08	43.36	67.50	30.79	33.23	32.41	+5.0%
<i>Qwen3-4B-Base</i>								
Base Model	5.10	3.65	21.33	31.90	10.85	17.54	15.06	–
GRPO	21.04	15.21	61.56	73.70	33.55	42.32	41.23	–
GRPO + CBS	20.31	18.44	63.05	74.45	34.47	43.73	42.41	+2.9%
GRPO + CBS*	20.52	16.56	62.19	75.55	34.93	43.43	42.20	+2.4%
DAPO	16.88	17.40	62.97	75.05	31.71	43.95	41.33	–
DAPO + CBS	20.62	20.94	62.89	75.85	36.58	46.70	43.93	+6.3%
DAPO + CBS*	18.96	17.81	63.83	76.30	34.83	44.88	42.77	+3.5%
GSPO	18.44	15.21	67.34	77.55	41.18	44.92	44.11	–
GSPO + CBS	21.46	18.33	67.03	76.05	36.49	44.18	43.92	-0.4%
GSPO + CBS*	21.25	19.17	66.02	77.85	34.83	46.55	44.28	+0.4%
<i>Qwen3-8B-Base</i>								
Base Model	8.33	6.56	35.78	53.25	19.67	26.22	24.97	–
GRPO	20.52	17.29	67.34	76.15	37.50	47.14	44.32	–
GRPO + CBS	23.65	20.83	69.45	77.45	38.79	48.59	46.46	+4.8%
GRPO + CBS*	26.46	19.48	69.92	77.45	38.88	47.59	46.63	+5.2%
DAPO	22.29	19.27	73.20	77.65	37.78	48.29	46.41	–
DAPO + CBS	23.54	21.88	75.55	78.25	39.71	49.41	48.06	+3.6%
DAPO + CBS*	24.38	19.58	68.52	76.85	37.50	46.96	45.63	-1.7%
GSPO	27.92	19.38	73.20	79.20	40.53	48.37	48.10	–
GSPO + CBS	23.02	20.73	73.83	83.10	45.13	49.93	49.29	+2.5%
GSPO + CBS*	25.10	20.52	72.34	79.65	42.00	51.08	48.45	+0.7%

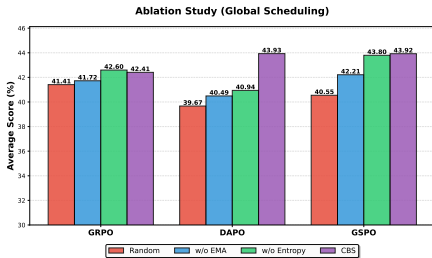


Figure 3. Ablation study results on Qwen3-4B-Base.

Table 3. Buffer-related runtime overhead (% of total time)

Scheduler	Method	1.7B	4B	8B
CBS	GRPO	3.83	1.45	1.15
	DAPO	2.88	1.37	1.00
	GSPO	1.97	1.79	1.56
CBS*	GRPO	1.70	0.92	0.84
	DAPO	1.95	0.83	0.69
	GSPO	1.98	0.90	0.79

following variants: (1) replacing the neural scheduler with random selection, denoted as *Random*; (2) replacing the EMA reward gain in Equation (10) with the raw reward gain (i.e., $V_{t+1} - V_t$), denoted as *w/o EMA*; and (3) removing the entropy penalty in Equation (5), denoted as *w/o Entropy*.

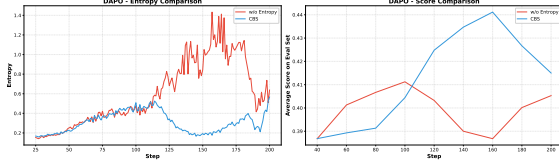


Figure 4. Comparison of Entropy and average score on the evaluation set for CBS and w/o Entropy.

As shown in Figure 3, CBS generally achieves superior performance compared to other variants, demonstrating the effectiveness of the proposed submodules. Notably, *Random* degrades the performance of all policy optimization methods, suggesting that data selection is non-trivial, as naive random sampling may introduce unstable or conflicting training signals. In Figure 4, we further compare the entropy dynamics of *w/o Entropy* and CBS. The results show that, without the entropy penalty, the entropy of *w/o Entropy* keeps increasing and the performance deteriorates accordingly, highlighting the necessity of the entropy penalty term. More ablation study results can be found in Appendix E.2.

Other experimental results. Due to space limitations, we put other experimental results in the Appendix, including hyperparameter sensitivity analysis (Appendix E.3) and scheduler pattern analysis (Appendix E.4).

7. Conclusion

In this paper, we present CBS, a contextual bandit-based scheduler that supports adaptive selection of high-value rollouts for both intra-group and global rollout scheduling problems. By encoding each rollout as a 10-dimensional training dynamics vector and using an MLP-based scheduler that learns adaptively from online policy feedback, CBS effectively prioritizes trajectories with high optimization potential, thereby significantly improving both final performance and training efficiency across three popular RLVR methods. Currently, CBS relies on a lightweight 10-dimensional representation of training dynamics for computational efficiency. Future work could explore integrating deep semantic representations from the LLM policy to uncover more nuanced selection patterns, further unlocking the potential of adaptive rollout scheduling.

Impact Statement

This paper presents an adaptive rollout scheduler whose goal is to advance the field of Reinforcement Learning with Verifiable Rewards. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

This research was supported by NSFC (No. 62276015), NSFC (No. 62506024) and GW2025-09.

References

- Arcuschin, I., Janiak, J., Krzyzanowski, R., Rajamanoharan, S., Nanda, N., and Conmy, A. Chain-of-thought reasoning in the wild is not always faithful. *CoRR*, abs/2503.08679, 2025.
- Azar, M. G., Guo, Z. D., Piot, B., Munos, R., Rowland, M., Valko, M., and Calandriello, D. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics*, 2024.
- Ban, Y., Yan, Y., Banerjee, A., and He, J. EE-net: Exploitation-exploration neural networks in contextual bandits. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=X_ch3VrNSRg.
- Ban, Y., Yan, Y., Banerjee, A., and He, J. Neural exploitation and exploration of contextual bandits. *CoRR*, abs/2305.03784, 2023.
- Ban, Y., Qi, Y., Wei, T., Liu, L., and He, J. Meta clustering of neural bandits. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (SIGKDD)*, 2024.
- Chen, H., Zhang, Y., Zhang, Q., Yang, H., Hu, X., Ma, X., Yanggong, Y., and Zhao, J. Maybe only 0.5% data is needed: A preliminary exploration of low training data instruction tuning. *CoRR*, abs/2305.09246, 2023.
- Chen, W., Wu, Y., Zhang, Z., Zhuang, F., He, Z., Xie, R., and Xia, F. Fairgap: Fairness-aware recommendation via generating counterfactual graph. *ACM Transactions on Information Systems*, 42(4):1–25, 2024.
- Chen, W., Yuan, M., Zhang, Z., Xie, R., Zhuang, F., Wang, D., and Liu, R. Fairdgl: Fairness-aware recommendation with dynamic graph contrastive learning. *IEEE Transactions on Knowledge and Data Engineering*, 2025a.
- Chen, W., Guo, X., Li, S., Zhang, Z., Zhong, Y., Zhuang, F., et al. Learning adaptive distribution alignment with neural characteristic function for graph domain adaptation. *arXiv preprint arXiv:2602.10489*, 2026a.
- Chen, W., Guo, X., Li, S., Zhong, Y., Zhang, Z., Zhuang, F., Liu, H., Zhang, L., Ye, G., and He, H. Learning structure-semantic evolution trajectories for graph domain adaptation. *arXiv preprint arXiv:2602.10506*, 2026b.

- Chen, Z., Ai, T., Li, Y., Li, G., Wei, Y., Zhou, W., Li, G., Yu, B., Chen, Z., Sun, H., Zhuang, F., Li, J., Wang, D., and Ban, Y. Llmboost: Make large language models stronger with boosting, 2025b. URL <https://arxiv.org/abs/2512.22309>.
- Christiano, P. F., Leike, J., Brown, T. B., Martic, M., Legg, S., and Amodei, D. Deep reinforcement learning from human preferences. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2017.
- Dai, J., Pan, X., Sun, R., Ji, J., Xu, X., Liu, M., Wang, Y., and Yang, Y. Safe RLHF: safe reinforcement learning from human feedback. In *International Conference on Learning Representations (ICLR)*, 2024.
- Dai, Z., Shu, Y., Verma, A., Fan, F. X., Low, B. K. H., and Jaillet, P. Federated neural bandits. In *International Conference on Learning Representations (ICLR)*, 2023.
- Das, N., Chakraborty, S., Pacchiano, A., and Chowdhury, S. R. Active preference optimization for sample efficient RLHF. In *Machine Learning and Knowledge Discovery in Databases. Research Track- European Conference, ECML PKDD*, volume 16017 of *Lecture Notes in Computer Science*, pp. 96–112. Springer, 2025.
- DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025.
- Dou, S., Wu, M., Xu, J., Zheng, R., Gui, T., and Zhang, Q. Improving RL exploration for LLM reasoning through retrospective replay. In *Natural Language Processing and Chinese Computing - 14th National CCF Conference, NLPCC 2025*, volume 16102 of *Lecture Notes in Computer Science*, pp. 594–606. Springer, 2025.
- He, C., Luo, R., Bai, Y., Hu, S., Thai, Z. L., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., Liu, J., Qi, L., Liu, Z., and Sun, M. Olympiadbench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- Hejna, J., Bhateja, C., Jiang, Y., Pertsch, K., and Sadigh, D. Re-mix: Optimizing data mixtures for large scale imitation learning. *CoRR*, abs/2408.14037, 2024.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- Huang, Y., Chen, H., Ruan, S., Zhang, Y., Wei, X., and Dong, Y. Mitigating overthinking in large reasoning models via manifold steering. *CoRR*, abs/2505.22411, 2025a.
- Huang, Z., Ban, Y., Fu, L., Li, X., Dai, Z., Li, J., et al. Adaptive batch-wise sample scheduling for direct preference optimization. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2025b.
- Iverson, H., Zhang, M., Brahman, F., Koh, P. W., and Dasigi, P. Large-scale data selection for instruction tuning. *CoRR*, abs/2503.01807, 2025.
- Jacot, A., Hongler, C., and Gabriel, F. Neural tangent kernel: Convergence and generalization in neural networks. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2018.
- Jaech, A., Kalai, A., Lerer, A., Richardson, A., El-Kishky, A., Low, A., Helyar, A., Madry, A., Beutel, A., Carney, A., Iftimie, A., Karpenko, A., Passos, A. T., Neitz, A., Prokofiev, A., Wei, A., Tam, A., Bennett, A., Kumar, A., Saraiva, A., Vallone, A., Duberstein, A., Kondrich, A., Mishchenko, A., Applebaum, A., Jiang, A., Nair, A., Zoph, B., Ghorbani, B., Rossen, B., Sokolowsky, B., Barak, B., McGrew, B., Minaiev, B., Hao, B., Baker, B., Houghton, B., McKinzie, B., Eastman, B., Lugaresi, C., Bassin, C., Hudson, C., Li, C. M., de Bourcy, C., Voss, C., Shen, C., Zhang, C., Koch, C., Orsinger, C., Hesse, C., Fischer, C., Chan, C., Roberts, D., Kappler, D., Levy, D., Selsam, D., Dohan, D., Farhi, D., Mely, D., Robinson, D., Tsipras, D., Li, D., Oprica, D., Freeman, E., Zhang, E., Wong, E., Proehl, E., Cheung, E., Mitchell, E., Wallace, E., Ritter, E., Mays, E., Wang, F., Such, F. P., Raso, F., Leoni, F., Tsimplouras, F., Song, F., von Lohmann, F., Sulit, F., Salmon, G., Parascandolo, G., Chabot, G., Zhao, G., Brockman, G., Leclerc, G., Salman, H., Bao, H., Sheng, H., Andrin, H., Bagherinezhad, H., Ren, H., Lightman, H., Chung, H. W., Kivlichan, I., O’Connell, I., Osband, I., Gilaberte, I. C., and Akkaya, I. Openai o1 system card. *CoRR*, abs/2412.16720, 2024.
- Jia, Y., Zhang, W., Zhou, D., Gu, Q., and Wang, H. Learning neural contextual bandits through perturbed rewards. In *International Conference on Learning Representations (ICLR)*, 2022.
- Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V. V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving quantitative reasoning problems with language models. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
- Li, Z., Liang, X., Tang, Z., Ji, L., Wang, P., Xu, H., W, X., Huang, H., Deng, W., Gong, Y., Guo, Z., Liu, X., Yin, F.,

- and Liu, C. Tldr: Too long, do re-weighting for efficient LLM reasoning compression. *CoRR*, abs/2506.02678, 2025.
- Li, Z., Tian, W., Ban, Y., Chen, J., Zhang, H., Liu, Y., and Zhuang, F. Counterfactual credit policy optimization for multi-agent collaboration. *CoRR*, abs/2603.21563, 2026a. doi: 10.48550/ARXIV.2603.21563. URL <https://doi.org/10.48550/arXiv.2603.21563>.
- Li, Z., Tian, W., Chen, J., Huang, K., Zhang, H., Yang, H., Ren, T., Jiang, J., Peng, Y., Ban, Y., and Zhuang, F. Adaptive robust estimator for multi-agent reinforcement learning. *CoRR*, abs/2603.21574, 2026b. doi: 10.48550/ARXIV.2603.21574. URL <https://doi.org/10.48550/arXiv.2603.21574>.
- Lin, Z., Wang, X., Cao, J., Chai, J., Wang, L., Lu, X., Lin, W., He, R., and Yin, G. Resrl: Boosting llm reasoning via negative sample projection residual reinforcement learning, 2026a. URL <https://arxiv.org/abs/2605.00380>.
- Lin, Z., Wang, X., Cao, J., Chai, J., Yin, G., Lin, W., and He, R. Rest: Reshaping token-level policy gradients for tool-use large language models. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL <https://openreview.net/forum?id=gNZlaKRWki>.
- Lin, Z., Wang, X., Yang, H., Chai, J., Cao, J., Yin, G., Lin, W., and He, R. Awpo: Enhancing tool-use of large language models through adaptive integration of reasoning rewards, 2026c. URL <https://arxiv.org/abs/2512.19126>.
- Liu, B., Wang, A., Min, Z., Yao, L., Zhang, H., Liu, Y., Zeng, A., and Su, J. Spec-rl: Accelerating on-policy reinforcement learning via speculative rollouts. *arXiv preprint arXiv:2509.23232*, 2025a.
- Liu, D., Yang, F., Wang, X., Yan, S., Chai, J., Li, J., Ban, Y., Mao, Z., Lin, W., and Yin, G. CDRRM: contrast-driven rubric generation for reliable and interpretable reward modeling. *CoRR*, abs/2603.08035, 2026. doi: 10.48550/ARXIV.2603.08035. URL <https://doi.org/10.48550/arXiv.2603.08035>.
- Liu, Z., Chen, C., Li, W., Qi, P., Pang, T., Du, C., Lee, W. S., and Lin, M. Understanding rl-zero-like training: A critical perspective. *CoRR*, abs/2503.20783, 2025b.
- Lu, K., Yuan, H., Yuan, Z., Lin, R., Lin, J., Tan, C., Zhou, C., and Zhou, J. #instag: Instruction tagging for analyzing supervised fine-tuning of large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- Mahmud, S., Nakamura, M., and Zilberstein, S. MAPLE: A framework for active preference learning guided by large language models. In Walsh, T., Shah, J., and Kolter, Z. (eds.), *AAAI Conference on Artificial Intelligence (AAAI)*, 2025.
- Mehta, V., Das, V., Neopane, O., Dai, Y., Bogunovic, I., Schneider, J. G., and Neiswanger, W. Sample efficient reinforcement learning from human feedback via active exploration. *CoRR*, abs/2312.00267, 2023.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E. J., and Hashimoto, T. sl: Simple test-time scaling. *CoRR*, abs/2501.19393, 2025.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P. F., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2022.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- Saunshi, N., Gupta, A., Braverman, M., and Arora, S. Understanding influence functions and datamodels via harmonic analysis. In *International Conference on Learning Representations (ICLR)*. OpenReview.net, 2023.
- Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024.
- Shen, J. H., Sharma, A., and Qin, J. Towards data-centric RLHF: simple metrics for preference dataset comparison. *CoRR*, abs/2409.09603, 2024.
- Sheng, G., Zhang, C., Ye, Z., Wu, X., Zhang, W., Zhang, R., Peng, Y., Lin, H., and Wu, C. Hybridflow: A flexible and efficient RLHF framework. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*, 2025.
- Wang, H., Li, X., Wang, D., Zhou, H., Huang, Z., Yang, Y., Li, J., and Ban, Y. Policy improvement reinforcement learning. *CoRR*, abs/2604.00860, 2026. doi: 10.48550/ARXIV.2604.00860. URL <https://doi.org/10.48550/arXiv.2604.00860>.

- Wang, Z., Awasthi, P., Dann, C., Sekhari, A., and Gentile, C. Neural active learning with performance guarantees. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2021.
- Xia, M., Malladi, S., Gururangan, S., Arora, S., and Chen, D. LESS: selecting influential data for targeted instruction tuning. In *International Conference on Machine Learning (ICML)*, 2024.
- Xu, Y. E., Savani, Y., Fang, F., and Kolter, Z. Not all rollouts are useful: Down-sampling rollouts in LLM reinforcement learning. *CoRR*, abs/2504.13818, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025a.
- Yang, F., Chen, Z., Wang, X., Lu, X., Chai, J., Yin, G., Lin, W., Ma, S., Zhuang, F., Wang, D., et al. Your group-relative advantage is biased. *CoRR*, abs/2601.08521, 2026.
- Yang, S., Dou, C., Guo, P., Lu, K., Ju, Q., Deng, F., and Xin, R. DCPO: dynamic clipping policy optimization. *CoRR*, abs/2509.02333, 2025b.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Dai, W., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W., Zhang, Y., Yan, L., Qiao, M., Wu, Y., and Wang, M. DAPO: an open-source LLM reinforcement learning system at scale. *CoRR*, abs/2503.14476, 2025a.
- Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Dai, W., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W., Zhang, Y., Yan, L., Qiao, M., Wu, Y., and Wang, M. DAPO: an open-source LLM reinforcement learning system at scale. *CoRR*, abs/2503.14476, 2025b.
- Yuan, M., Xiao, Y., Chen, W., Zhao, C., Wang, D., and Zhuang, F. Hyperbolic diffusion recommender model. In *Proceedings of the ACM on Web Conference 2025*, pp. 1992–2006, 2025.
- Yue, Y., Chen, Z., Lu, R., Zhao, A., Wang, Z., Yue, Y., Song, S., and Huang, G. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model? *CoRR*, abs/2504.13837, 2025. doi: 10.48550/ARXIV.2504.13837. URL <https://doi.org/10.48550/arXiv.2504.13837>.
- Yun, C., Sra, S., and Jadbabaie, A. Small relu networks are powerful memorizers: a tight analysis of memorization capacity. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2019.
- Zhan, R., Li, Y., Wang, Z., Qu, X., Liu, D., Shao, J., Wong, D. F., and Cheng, Y. Exgrpo: Learning to reason from experience. *CoRR*, abs/2510.02245, 2025.
- Zhang, H., Fu, J., Zhang, J., Fu, K., Wang, Q., Zhang, F., and Zhou, G. RLEP: reinforcement learning with experience replay for LLM reasoning. *CoRR*, abs/2507.07451, 2025.
- Zhang, W., Zhou, D., Li, L., and Gu, Q. Neural thompson sampling. In *International Conference on Learning Representations (ICLR)*, 2021.
- Zhang, Z., Huang, Z., Xia, X., Wang, D., Zhuang, F., Ma, S., Ding, N., Yang, Y., Li, J., and Ban, Y. Heterogeneous agent collaborative reinforcement learning. *CoRR*, abs/2603.02604, 2026. doi: 10.48550/ARXIV.2603.02604. URL <https://doi.org/10.48550/arXiv.2603.02604>.
- Zheng, C., Liu, S., Li, M., Chen, X., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., and Lin, J. Group sequence policy optimization. *CoRR*, abs/2507.18071, 2025a.
- Zheng, H., Zhou, Y., Bartoldson, B. R., Kailkhura, B., Lai, F., Zhao, J., and Chen, B. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts. *CoRR*, abs/2506.02177, 2025b.
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., Zhang, S., Ghosh, G., Lewis, M., Zettlemoyer, L., and Levy, O. LIMA: less is more for alignment. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- Zhou, D., Li, L., and Gu, Q. Neural contextual bandits with ucb-based exploration. In *International Conference on Machine Learning (ICML)*, 2020.
- Zhu, X., Xia, M., Wei, Z., Chen, W., Chen, D., and Meng, Y. The surprising effectiveness of negative reinforcement in LLM reasoning. *CoRR*, abs/2506.01347, 2025.
- Zou, J., Ban, Y., Li, Z., Qi, Y., Qiu, R., Yang, L., and He, J. Transformer copilot: Learning from the mistake log in LLM fine-tuning. In *The Thirty-ninth Annual*

Conference on Neural Information Processing Systems,
2025. URL <https://openreview.net/forum?id=MRvx1TlkNQ>.

A. Introduction of Group Relative Policy Optimization Algorithms

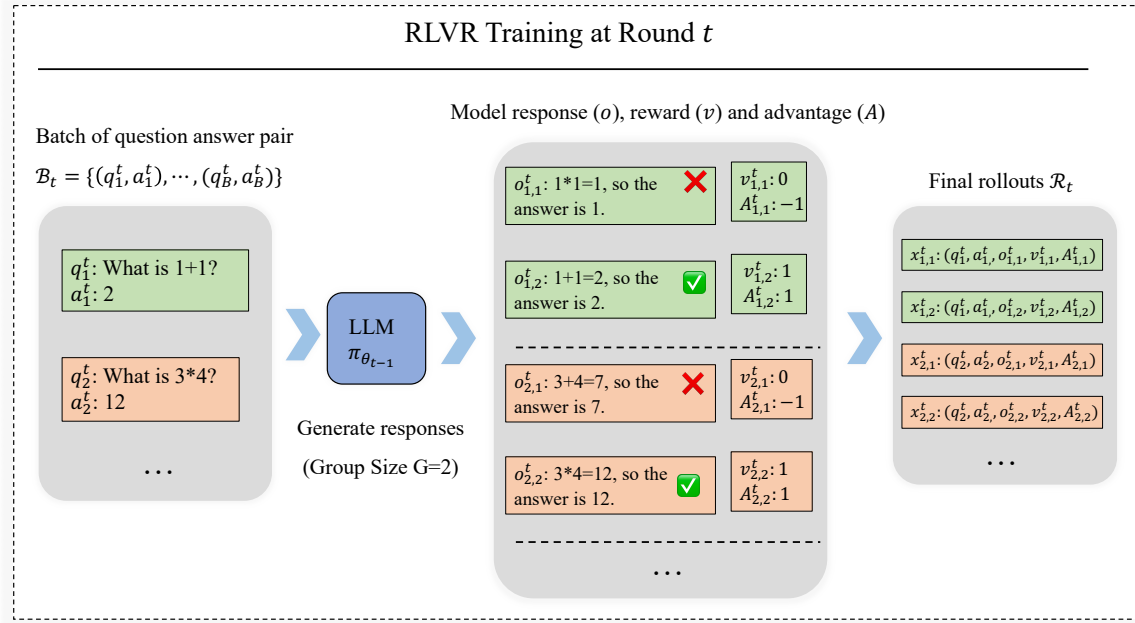


Figure 5. Pipeline of RLVR training. At round t , a batch of prompt-answer pairs $\mathcal{B}_t = \{(q_i^t, a_i^t)\}_{i \in [B]}$ is first sampled. The current policy $\pi_{\theta_{t-1}}$ then generates a group of responses $\{o_{i,j}^t\}_{j \in [G]}$ for each question q_i^t . The corresponding rewards $\{v_{i,j}^t\}_{j \in [G]}$ and advantages $\{A_{i,j}^t\}_{j \in [G]}$ are computed according to responses $\{o_{i,j}^t\}_{j \in [G]}$ and answers a_i^t , and the resulting rollout instances together form $\mathcal{R}_t = \{x_{i,j}^t\}_{i \in [B], j \in [G]}$.

A.1. Target of Different Policy Optimization Methods

Reinforcement Learning is a technique that has been widely applied across many domains (Chen et al., 2024; 2025a; 2026a;b; Yuan et al., 2025). Recently, RLVR has become a critical method for improving the reasoning ability of LLMs (Lin et al., 2026a;c;b; Wang et al., 2026; Li et al., 2026b;a; Zhang et al., 2026; Liu et al., 2026). In this section, we introduce three representative policy optimization methods.

GRPO (Group Relative Policy Optimization) is a popular method for policy optimization in language model post-training. It optimizes the policy π_θ by maximizing

$$\mathcal{J}_\theta = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min(A_i r_{i,t}(\theta), A_i \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})) \right], \quad (14)$$

where the $\text{clip}(r, p_{\text{low}}, p_{\text{high}}) = \max(p_{\text{low}}, \min(r, p_{\text{high}}))$ is a hard clip function. The GRPO can be considered as a variant of Equation (1) by setting Φ to

$$\Phi(A, r_{i,t}(\theta), \epsilon_{\text{low}}, \epsilon_{\text{high}}) = \min(A_i r_{i,t}(\theta), A_i \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})). \quad (15)$$

Dynamic Sampling Policy Optimization (DAPO) introduces four key improvements over GRPO: 1) *Dynamic Sampling*, which filters out zero-advantage responses; 2) *Clip Higher*, which increases the clip upper bound ϵ_{high} ; 3) *Overlong Filter*, which shapes the reward by penalizing overly long responses; and 4) *Token-Level Loss*, which normalizes the loss by the total token length across the group of responses. By combining these components, the policy π_θ is optimized by maximizing

$$\mathcal{J}_\theta = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(A_i r_{i,t}(\theta), \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) A_i \right) \right] \quad (16)$$

s.t. $0 < |\{o_i | \text{is_equivalent}(a, o_i)\}| < G$

Algorithm 1 Policy Training with Data Scheduler

```

1: Input: policy  $\pi_{\theta_0}$ , scheduler  $s_{\phi_0}$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $\nabla$  Rollout Construction
4:   Randomly select a batch  $\mathcal{B}_t$  from training data  $\mathcal{D}$ 
5:   Generate rollout  $\mathcal{R}_t$  by current policy  $\pi_{\theta_{t-1}}$  and construct corresponding representations in Table 1
6:   Construct  $\mathcal{C}_t$  using  $\mathcal{R}_t$  and  $\mathcal{C}_{t-1}$ 
7:    $\nabla$  Scheduler Training
8:   Construct sample reward  $\hat{R}(x, \theta_{t-2}, \theta_{t-1})$  for  $x \in \bar{\mathcal{C}}_{t-1}$  according to Equation (10) and Equation (11)
9:   Update the scheduler from  $s_{\phi_{t-2}}$  to  $s_{\phi_{t-1}}$  according to Equation (7)
10:   $\nabla$  Data Scheduling
11:  Select  $\bar{\mathcal{C}}_t$  from  $\mathcal{C}_t$  according to Equation (8)
12:  Update  $\pi_{\theta_{t-1}}$  to  $\pi_{\theta_t}$  according to Equation (2)
13:   $\nabla$  Arm Representation Update
14:  Update representations associated to the data in  $\mathcal{C}_t$  if needed
15: end for
    
```

The policy optimization function of DAPO can be considered as the same Equation (1) variant of GRPO.

Group Sequence Policy Optimization (GSPO) adopts the sequence-level clip rather than the token-level clip, specifically, it optimizes the policy by maximizing

$$\mathcal{J}_\theta = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \min \left(A_i s_i(\theta), A_i \text{clip}(s_i(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \right) \right], \quad (17)$$

where $s_i(\theta) = \exp \left(\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t}|q, o_{i,<t})} \right)$ is the normalized sequence-level importance sampling weight. Equation (17) can also be written into a token form (Zheng et al., 2025a) of

$$\mathcal{J}_\theta = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i^t|} \sum_{t=1}^{|o_i^t|} \min(A_i s_{i,t}(\theta), A_i \text{clip}(s_{i,t}(\theta), 1 - \epsilon_{\text{min}}, 1 + \epsilon_{\text{max}})) \right], \quad (18)$$

where $s_{i,t}(\theta) = \frac{\text{sg}[s_i(\theta)] * \pi_{\text{old}}(y_{i,t}|x_i, y_{i,<t})}{\text{sg}(\pi_\theta(y_{i,t}|x_i, y_{i,<t}))} \cdot r_{i,t}(\theta)$ and sg indicates the stop gradient operation. Let $w_{i,t}$ indicates $\frac{\text{sg}[s_i(\theta)] * \pi_{\text{old}}(y_{i,t}|x_i, y_{i,<t})}{\text{sg}(\pi_\theta(y_{i,t}|x_i, y_{i,<t}))}$. The GSPO can be considered as a variant of Equation (1) by setting Φ to

$$\Phi(A, r_{i,t}(\theta), \epsilon_{\text{low}}, \epsilon_{\text{high}}) = \min(A_i \cdot w_{i,t} \cdot r_{i,t}(\theta), A_i \text{clip}(w_{i,t} \cdot r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}})). \quad (19)$$

A.2. Training Pipeline

We provide a visual illustration for the RLVR training pipeline in Figure 5.

B. Proof of Theorems

B.1. Definitions

We instantiate the selector network $s_\phi(\cdot)$ as a fully connected network with depth $L \geq 2$ and width m , where $s_\phi(h) = \mathbf{W}_L \sigma(\mathbf{W}_{L-1} \sigma(\mathbf{W}_{L-2} \cdots \sigma(\mathbf{W}_1 h)))$ with $\sigma(\cdot)$ being ReLU activation, $\mathbf{W}_1 \in \mathbb{R}^{m \times 10}$, $\mathbf{W}_i \in \mathbb{R}^{m \times m}$ for $1 \leq i \leq L-1$, $\mathbf{W}_L \in \mathbb{R}^{1 \times m}$, and $\phi = [\text{vec}(\mathbf{W}_L), \dots, \text{vec}(\mathbf{W}_1)]$. For initialization, entries of $\mathbf{W}_L$ are drawn independently from $\mathcal{N}(0, 1/m)$, and for $1 \leq i \leq L-1$, entries of each $\mathbf{W}_i$ are drawn independently from $\mathcal{N}(0, 2/m)$. We slightly abuse the notations and rewrite the arm representations $\{h_i^t\}_{i \in [M], t \in [T]}$ and rewards $\{r_i^t\}_{t \in [T], i \in [M]}$ into $\{h_i\}_{i \in [TM]}$ and $\{r_i\}_{i \in [TM]}$ respectively. The NTK matrix is defined as below,

Definition B.1 (NTK (Jacot et al., 2018; Wang et al., 2021)). Let $\mathcal{N}$ denote the normal distribution. Given the data instances $\{h_t\}_{t=1}^{TM}$, for all $i, j \in [TM]$, define:

$$\begin{aligned} H_{i,j}^0 &= \Sigma_{i,j}^0 = \langle h_i, h_j \rangle, \quad A_{i,j}^l = \begin{pmatrix} \Sigma_{i,i}^l & \Sigma_{i,j}^l \\ \Sigma_{j,i}^l & \Sigma_{j,j}^l \end{pmatrix}, \\ \Sigma_{i,j}^l &= 2\mathbb{E}_{a,b \sim \mathcal{N}(0, A_{i,j}^{l-1})} [\sigma(a)\sigma(b)], \\ H_{i,j}^l &= 2H_{i,j}^{l-1} \mathbb{E}_{a,b \sim \mathcal{N}(0, A_{i,j}^{l-1})} [\sigma'(a)\sigma'(b)] + \Sigma_{i,j}^l. \end{aligned}$$

Then the NTK matrix is defined as $H = (H^L + \Sigma^L)/2$

B.2. Lemma on Bounded Deviation

Given parameters θ_{t-1} and θ_t , the terms V_t and V_{t+1} can be viewed as stochastic estimates of the expected performance of θ_{t-1} and θ_t on the training set $\mathcal{D}$ (and similarly for E_t and E_{t+1}). The following lemma shows that the deviation between the stochastic estimate and its corresponding expectation is bounded.

Lemma B.2. *Let $R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t)$ denote the reward in (5). Assume the per-sample reward satisfies $v_i^t \in \{0, 1\}$ for all t, i , and let $N = |\mathcal{V}|$ be the vocabulary size. Then*

$$|\mathbb{E}[R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t)] - R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t)| \leq 2(1 + w_e \log N), \quad (20)$$

Proof. Since $v_i^t \in \{0, 1\}$, we have $0 \leq V_t = \text{mean}(\{v_i^t\}) \leq 1$, hence

$$V_{t+1} - V_t \in [-1, 1]. \quad (21)$$

For the entropy term, for any context the Shannon entropy of a distribution over a vocabulary of size N is bounded by

$$0 \leq H(\pi_{\theta_{t-1}}(\cdot | \cdot)) \leq \log N. \quad (22)$$

By definition, e_i^t is the average of token-level entropies, so $e_i^t \in [0, \log N]$, and thus

$$E_t = \text{mean}(\{e_i^t\}) \in [0, \log N], \quad E_{t+1} - E_t \in [-\log N, \log N]. \quad (23)$$

Moreover, $\mathbb{I}[E_t > e_{\min}] \in \{0, 1\}$, so

$$-w_e \mathbb{I}[E_t > e_{\min}] (E_{t+1} - E_t) \in [-w_e \log N, w_e \log N]. \quad (24)$$

Therefore,

$$R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) \in \left[-(1 + w_e \log N), 1 + w_e \log N \right]. \quad (25)$$

For any random variable $X \in [a, b]$, we have $|\mathbb{E}[X] - X| \leq b - a$. Applying this with $a = -(1 + w_e \log N)$ and $b = 1 + w_e \log N$ yields

$$|\mathbb{E}[R] - R| \leq 2(1 + w_e \log N), \quad (26)$$

which proves the claim. $\square$

B.3. Lemma for Neural Regression

Lemma B.3 (Theorem 3.1 of (Yun et al., 2019)). *Given any dataset $\{(x_i, y_i)\}_{i=1}^N$ that all x_i are different and all $y_i \in [-1, 1]$. If $L \geq 3$ and $4 \lfloor \frac{m}{2} \rfloor \lfloor \frac{m}{2} \rfloor \geq N$, there exist a ϕ' such that $s_{\phi'}(x_i) = y_i$ for all $i \in [N]$.*

Lemma B.4 (Lemma B.1 of (Ban et al., 2023)). *Let $\mathcal{H} = \{(h_j, r_j)\}_{j=1}^N$ be a finite feature-label set satisfying $\|h_i\|_2 = 1$ and $r_i \in [0, 1]$ for any $i \in [N]$. For any $R > 0$ and $\delta \in (0, 1)$, if $m \geq \Omega(\text{poly}(R, L, N, \log(1/\delta)))$, then with probability at least $1 - \delta$ over the initialization ϕ_0 , the following bounds hold uniformly for all $(h, r) \in \mathcal{H}$ and all $\phi \in B_R(\phi_0) = \{\phi : \|\phi - \phi_0\| \leq R/m^{1/4}\}$:*

$$|s_\phi(h)| \leq O(1), \quad \|\nabla_\phi s_\phi(h)\|_2 \leq O(\sqrt{L}), \quad \|\nabla_\phi \ell(\phi; h, r)\|_2 \leq O(\sqrt{L}) \quad (27)$$

Lemma B.5 (Uniform local perturbation bound). *Let $\mathcal{H} = \{(h_j, r_j)\}_{j=1}^N$ be a finite feature-label set satisfying $\|h_i\|_2 = 1$ and $r_i \in [0, 1]$ for any $i \in [N]$, and let $\ell(\phi; h, r) = \frac{1}{2}(s_\phi(h) - r)^2$. For any $R > 0$ and $\delta \in (0, 1)$, if $m \geq \Omega(\text{poly}(R, L, N, \log(1/\delta)))$, then with probability at least $1 - \delta$ over the initialization ϕ_0 , the following bounds hold uniformly for all $(h, r) \in \mathcal{H}$ and all $\phi, \phi' \in B_R(\phi_0) = \{\phi : \|\phi - \phi_0\| \leq R/m^{1/4}\}$:*

$$|s_\phi(h) - s_{\phi'}(h)| \leq O\left(\frac{R\sqrt{L}}{m^{1/4}}\right). \quad (28)$$

Proof. Let $\rho = R/m^{1/4}$ and choose $\omega_\star = c_0 L^{-9/2}(\log m)^{-3}$, where $c_0 > 0$ is a sufficiently small universal constant. For $m \geq \Omega(\text{poly}(R, L, N, \log(1/\delta)))$, we have $\rho \leq \omega_\star$ and $O(NL^2) \exp(-\Omega(m\omega_\star^{2/3}L)) \leq \delta/2$. Hence, $B_R(\phi_0)$ is contained in the local ball of radius $\omega_\star$. By Lemma B.2 of (Ban et al., 2023), uniformly for all $(h, r) \in \mathcal{H}$ and all $\phi, \phi' \in B_R(\phi_0)$,

$$|s_\phi(h) - s_{\phi'}(h) - \langle \nabla_\phi s_{\phi'}(h), \phi - \phi' \rangle| \leq O(\omega_\star^{1/3} L^2 \sqrt{\log m}) \|\phi - \phi'\|_2.$$

Since $\omega_\star^{1/3} L^2 \sqrt{\log m} \leq O(\sqrt{L})$, this gives

$$|s_\phi(h) - s_{\phi'}(h) - \langle \nabla_\phi s_{\phi'}(h), \phi - \phi' \rangle| \leq O(\sqrt{L}) \|\phi - \phi'\|_2.$$

By Lemma B.4, $\|\nabla_\phi s_{\phi'}(h)\|_2 \leq O(\sqrt{L})$ with probability at least $1 - \delta/2$. Taking a union bound over the two events, with probability at least $1 - \delta$,

$$\begin{aligned} |s_\phi(h) - s_{\phi'}(h)| &\leq |\langle \nabla_\phi s_{\phi'}(h), \phi - \phi' \rangle| + O(\sqrt{L}) \|\phi - \phi'\|_2 \\ &\leq O(\sqrt{L}) \|\phi - \phi'\|_2 \leq O\left(\frac{R\sqrt{L}}{m^{1/4}}\right), \end{aligned}$$

where the last step uses $\|\phi - \phi'\|_2 \leq 2R/m^{1/4}$. $\square$

Lemma B.6 (Lemma 5.2 of (Ban et al., 2023)). *Let $\mathcal{H} = \{(h_j, r_j)\}_{j=1}^N$ be a finite feature-label set satisfying $\|h_i\|_2 = 1$ and $r_i \in [0, 1]$ for any $i \in [N]$. For any arm representations $\{h_i\}_{i \in [TM]}$ and rewards $\{r_i\}_{i \in [TM]}$ sampled from $\mathcal{H}$ and satisfying Assumption 5.2, if $m \geq \Omega\left(\text{poly}(T, R, L, N, \log(\delta), \frac{1}{\lambda_0})\right)$, with probability at least $1 - \delta$ over initialization of ϕ_0 , there exists $\phi' \in B(\phi_0, \tilde{\Omega}(T^{3/2}))$ such that*

$$\sum_{t=1}^{TM} (s_{\phi'}(h_t) - r_t)^2 \leq \tilde{O}\left(\sqrt{\tilde{d}} + S\right)^2 \ast \tilde{d}, \quad (29)$$

where $S = \sqrt{r^T H^{-1} r}$ and $\tilde{d} = \frac{\log \det(I+H)}{\log(1+TM)}$.

B.4. Proof of Theorem 5.3

Let $\phi^\star$ denote the optimal policy parameter for buffer size jBG . To prove Theorem 5.3, we can show that there exists a policy parameter ϕ' such that, for any given rollout sequence $\mathcal{R} = \{\mathcal{R}_1, \dots, \mathcal{R}_T\}$, the selected data sequence $x_1(\phi', \mathcal{R}, iBG), \dots, x_T(\phi', \mathcal{R}, iBG)$ is identical to the sequence $x_1(\phi^\star, \mathcal{R}, jBG), \dots, x_T(\phi^\star, \mathcal{R}, jBG)$, where $x_t(\phi, \mathcal{R}, M)$ represents the data chosen by scheduler s_ϕ when the buffer size is M and the rollout data is $\mathcal{R}$. Therefore, we can write the following inequality:

$$\varphi(iBG) = \max_{\phi} \mathbb{E}_{M=iBG} [\max_{t \in T} g(\theta_t)] \geq \mathbb{E}_{\phi=\phi', M=iBG} [\max_{t \in T} g(\theta_t)] = \varphi(jBG),$$

where $\mathbb{E}_{\phi=\phi', M=iBG} [\max_{t \in T} g(\theta_t)]$ represents the optimal performance when the buffer size is iBG and the scheduler parameter is ϕ' .

Next, we describe how to construct ϕ' based on $\phi^\star$. Since the vocabulary size $|V|$, the response length $L_{\max}$, and the number of training rounds T are finite, all possible rollout data and arm representations come from a finite set. Let $H \subset \mathbb{R}^{10}$ denote

the set of all possible arm representations, and let $U = \max_{h \in H} |s_{\phi^*}(h)|$ be the upper bound of the absolute value of scores, with $C = |H|$ representing the size of the arm set. We assign a label y_h for each $h \in H$ as follows:

$$y_h = \frac{2U \cdot \mathbb{I}[h_s < j] + s_{\phi^*}(h)}{3U},$$

where h_s denotes the delta step feature for the arm representations in Table 1. By setting $L = 3$, $m = 2C$, and applying Lemma B.3, we obtain $s_{\phi'}(h) = y_h$ for all $h \in H$.

Now, consider the scores $s_{\phi'}(h), s_{\phi'}(h')$ for any two data points $x, x' \in \mathcal{F}_t$, where h, h' are the representations for x and x' , respectively. The buffer $\mathcal{F}_t = \cup_{i=t-i+1}^t \mathcal{R}_i$ includes the data from the latest i steps. For data within the latest j steps, the delta step $h_s < j$, so y_h will be larger than the labels for data older than the latest j steps according to the definition of y_h . For two data points both within the latest j steps, we have:

$$s_{\phi'}(h) - s_{\phi'}(h') = \frac{s_{\phi^*}(h) - s_{\phi^*}(h')}{3U}.$$

Thus, the relative difference in scores is the same as for s_{ϕ^*} . Combining the two situations together, the strategy $s_{\phi'}$ behaves identically to s_{ϕ^*} . As a result, for any rollout sequence $\mathcal{R} = \{\mathcal{R}_1, \dots, \mathcal{R}_T\}$, the data selection sequence of $s_{\phi'}$ will be the same as that of s_{ϕ^*} . This completes the proof.

B.5. Proof of Theorem 5.4

Proof. For notation simplicity, write $g_a^t = g(\mathcal{A}(x_a^t, \theta_{t-1}))$. Let $a_t^* = \arg \max_{a \in [M]} g_a^t$, $a_t^g = \arg \max_{a \in [M]} s_{\phi_{t-1}}(h_a^t)$, and let a_t^e denote the exploration arm. Let $e_t \sim \text{Bernoulli}(\epsilon_t)$ indicate whether exploration is performed, so $a_t = a_t^e$ if $e_t = 1$ and $a_t = a_t^g$ otherwise. Then

$$\begin{aligned} \mathbb{E}[R_T] &= \sum_{t=1}^T \mathbb{E} \left[g_{a_t^*}^t - g_{a_t}^t \right] \\ &= \sum_{t=1}^T \mathbb{E} \left[g_{a_t^*}^t - g_{a_t^g}^t \right] + \sum_{t=1}^T \mathbb{E} \left[e_t \left(g_{a_t^g}^t - g_{a_t^e}^t \right) \right] \\ &\stackrel{(a)}{\leq} \sum_{t=1}^T \mathbb{E} \left[g_{a_t^*}^t - g_{a_t^g}^t \right] + \sum_{t=1}^T \mathbb{E} [e_t] \\ &\leq \sum_{t=1}^T \mathbb{E} \left[g_{a_t^*}^t - g_{a_t^g}^t \right] + \sum_{t=1}^T \epsilon_t, \end{aligned}$$

where (a) holds because the reward range is $[0, 1]$. By setting $T_w = 1$, $\epsilon_0 = 1$, $\Delta_\epsilon = T^{-1/2}$, and $\epsilon_{\min} = 0$, the exploration probability in Equation (9) becomes $\epsilon_1 = 1$ and, for $t \geq 2$, $\epsilon_t = \max(1 - (t-1)/\sqrt{T}, 0)$. Therefore,

$$\begin{aligned} \sum_{t=1}^T \epsilon_t &= 1 + \sum_{t=2}^T \max \left(1 - \frac{t-1}{\sqrt{T}}, 0 \right) \\ &= 1 + \sum_{k=1}^{T-1} \max \left(1 - \frac{k}{\sqrt{T}}, 0 \right), \end{aligned}$$

where $k = t-1$. The summand is positive only when $k < \sqrt{T}$, so there are at most $\lceil \sqrt{T} \rceil - 1$ nonzero terms in the second summation. Since each summand is at most 1,

$$\sum_{t=1}^T \epsilon_t \leq 1 + (\lceil \sqrt{T} \rceil - 1) \leq 1 + \sqrt{T} \leq 2\sqrt{T}.$$

Using $s_{\phi_{t-1}}(h_{a_t^g}^t) \geq s_{\phi_{t-1}}(h_{a_t^*}^t)$, for any auxiliary parameter sequence $\{\phi_t^*\}_{t=0}^T$, we have

$$\begin{aligned} \sum_{t=1}^T \mathbb{E} [g_{a_t^*}^t - g_{a_t^g}^t] &\stackrel{(a)}{=} \sum_{t=1}^T \mathbb{E} [r_{a_t^*}^t - r_{a_t^g}^t] \\ &\leq \underbrace{\sum_{t=1}^T \mathbb{E} [r_{a_t^*}^t - s_{\phi_{t-1}}^*(h_{a_t^*}^t)]}_{I_1} + \underbrace{\sum_{t=1}^T \mathbb{E} [s_{\phi_{t-1}}^*(h_{a_t^*}^t) - s_{\phi_{t-1}}(h_{a_t^*}^t)]}_{I_2} + \underbrace{\sum_{t=1}^T \mathbb{E} [s_{\phi_{t-1}}(h_{a_t^*}^t) - g_{a_t^g}^t]}_{I_3}, \end{aligned}$$

where (a) holds because $g_a^t = \mathbb{E}[r_a^t | \mathcal{G}_t]$, where $\mathcal{G}_t$ denotes the information available before observing the round- t rewards. By the law of iterated expectation and the fact that a_t^* and a_t^g are determined by $\mathcal{G}_t$, we have $\mathbb{E}[r_{a_t^*}^t - r_{a_t^g}^t] = \mathbb{E}[g_{a_t^*}^t - g_{a_t^g}^t]$.

We now define the auxiliary trajectory used in I_1 and I_2 . Let $\phi_0^* = \phi_0$ and $\ell(\phi, h, r) = \frac{1}{2}(s_\phi(h) - r)^2$, we define $\phi_t^* = \phi_{t-1}^* - \eta_2 \nabla_\phi \ell(\phi_{t-1}^*, h_{a_t^*}^t, r_{a_t^*}^t)$. The actual scheduler trajectory can be written as $\phi_t = \phi_{t-1} - \eta_2 \nabla_\phi \ell(\phi_{t-1}; h_t^{\text{obs}}, r_t^{\text{obs}})$ for some observed feature-label pair $(h_t^{\text{obs}}, r_t^{\text{obs}})$. Since the initial checkpoint, training set, vocabulary, maximum response length, and training horizon are fixed, and the rollout randomness has finite support over T rounds, all possible scheduler feature-label pairs are sampled from a finite set $\mathcal{H}_{\text{all}} = \{(h_j, r_j)\}_{j=1}^{N_{\mathcal{H}}}$ with $\|h_j\|_2 = 1$ and $r_j \in [0, 1]$. Let $\mathcal{E}_1$, $\mathcal{E}_2$, and $\mathcal{E}_3$ be the events in Lemma B.4, Lemma B.5, and Lemma B.6 with failure probabilities δ_1 , δ_2 , and δ_3 , respectively. Set $\delta_1 = \delta_2 = \delta_3 = \delta/3$, $R = \tilde{\Omega}(T^{3/2})$, and $\eta_2 = R^2/\sqrt{m}$. By a union bound, $\mathcal{E} = \mathcal{E}_1 \cap \mathcal{E}_2 \cap \mathcal{E}_3$ holds with probability at least $1 - \delta$ over ϕ_0 . In the following, we condition on $\mathcal{E}$.

On $\mathcal{E}_1$, both $\{\phi_t\}_{t=0}^T$ and $\{\phi_t^*\}_{t=0}^T$ stay in $B_R(\phi_0)$. Indeed, by Lemma B.4, $\|\nabla_\phi \ell(\phi; h, r)\|_2 \leq O(\sqrt{L})$ uniformly over $(h, r) \in \mathcal{H}_{\text{all}}$ and $\phi \in B_R(\phi_0)$. Therefore, for either trajectory,

$$\|\phi_t - \phi_0\|_2 \leq \sum_{\tau=1}^T \eta_2 O(\sqrt{L}) \leq O\left(\frac{TR^2\sqrt{L}}{\sqrt{m}}\right) \leq \frac{R}{m^{1/4}},$$

provided that $m \geq \Omega(T^4 R^4 L^2)$. The same argument applies to ϕ_t^* . On $\mathcal{E}_2$, by Lemma B.6 and the choice $R = \tilde{\Omega}(T^{3/2})$, there exists $\bar{\phi} \in B_R(\phi_0)$ such that

$$\mathcal{Q}_T := \sum_{t=1}^T \sum_{a=1}^M (s_{\bar{\phi}}(h_a^t) - r_a^t)^2 \leq \tilde{O}\left(\sqrt{\tilde{d}} + S\right)^2 \tilde{d}.$$

Finally, on $\mathcal{E}_2$, for any $\phi, \phi' \in B_R(\phi_0)$ and any $(h, r) \in \mathcal{H}_{\text{all}}$,

$$|s_\phi(h) - s_{\phi'}(h)| \leq \Delta_m, \quad \Delta_m = O\left(\frac{R\sqrt{L}}{m^{1/4}}\right).$$

The same lower bound $m \geq \Omega(T^4 R^4 L^2)$ also gives $T\Delta_m = O(1)$.

We now bound the three terms. For I_3 ,

$$\begin{aligned} I_3 &= \sum_{t=1}^T \mathbb{E} \left[|s_{\phi_{t-1}}(h_{a_t^g}^t) - s_{\bar{\phi}}(h_{a_t^g}^t) + s_{\bar{\phi}}(h_{a_t^g}^t) - r_{a_t^g}^t| \right] \\ &\leq \sum_{t=1}^T \mathbb{E} \left[|s_{\bar{\phi}}(h_{a_t^g}^t) - r_{a_t^g}^t| \right] + T\Delta_m \\ &\stackrel{(a)}{\leq} \sqrt{T} \mathbb{E} \left[\sqrt{\sum_{t=1}^T (s_{\bar{\phi}}(h_{a_t^g}^t) - r_{a_t^g}^t)^2} \right] + O(1) \\ &\leq \sqrt{T} \mathbb{E} [\sqrt{\mathcal{Q}_T}] + O(1) \\ &\leq \mathbb{E} [\tilde{O}(\sqrt{T}(\tilde{d} + S\sqrt{\tilde{d}}))] + O(1), \end{aligned}$$

where (a) comes by applying the Cauchy–Schwarz inequality. The same argument gives the bound for I_1 ,

$$\begin{aligned} I_1 &\leq \sum_{t=1}^T \mathbb{E} \left[\left| s_{\bar{\phi}}(h_{a_t^*}^t) - r_{a_t^*}^t \right| \right] + T \Delta_m \\ &\leq \sqrt{T} \mathbb{E} \left[\sqrt{\mathcal{Q}_T} \right] + O(1) \\ &\leq \mathbb{E} \left[\tilde{O} \left(\sqrt{T} \left(\tilde{d} + S \sqrt{\tilde{d}} \right) \right) \right] + O(1). \end{aligned}$$

For I_2 , since both ϕ_{t-1}^* and ϕ_{t-1} lie in $B_R(\phi_0)$, Lemma B.5 directly yields $I_2 \leq \sum_{t=1}^T \Delta_m = O(1)$.

Combining the bounds above, if $m \geq \Omega(\text{poly}(N_{\mathcal{H}}, T, R, L, \log(\frac{1}{\delta}), \frac{1}{\lambda_0}))$, with probability at least $1 - \delta$ over the scheduler initialization, we have

$$\begin{aligned} \mathbb{E}[R_T] &\leq I_1 + I_2 + I_3 + \sum_{t=1}^T \epsilon_t \\ &\leq \mathbb{E} \left[\tilde{O} \left(\sqrt{T} \left(\tilde{d} + S \sqrt{\tilde{d}} \right) + \sqrt{T} \right) \right]. \end{aligned}$$

This completes the proof. $\square$

C. Discussion on Subset Selection

We first justify the subset selection as sample scoring using a first-order Taylor expansion, and then discuss the regret bound under subset selection.

Justification of Sample Selection. When θ_{t-1} is fixed, the expression

$$R(\bar{\mathcal{C}}_t, \theta_{t-1}, \theta_t) = V_{t+1} - V_t - w_e \mathbb{I}[E_t > e_{\min}](E_{t+1} - E_t) \quad (30)$$

can be interpreted as a stochastic estimate of the function $f(\theta_t)$, which is defined as

$$f(\theta_t) = \mathbb{E}_{(q,a) \sim D, o \sim \pi_{\theta_t}(\cdot|q)} \left[v(a, o) - V_t - w_e \mathbb{I}[E_t > e_{\min}] \cdot \left(\sum_{x \in \mathcal{V}} \frac{\pi_{\theta_t}(x|q, o_{<t}) \log \pi_{\theta_t}(x|q, o_{<t})}{|o|} - E_t \right) \right], \quad (31)$$

where $v(\cdot)$ is the reward function of RLVR. Applying the first-order Taylor expansion around the reference point θ_{t-1} , we will get

$$f(\theta_t) \approx f(\theta_{t-1}) + \nabla_{\theta_{t-1}} f(\theta) \cdot (\theta_t - \theta_{t-1}). \quad (32)$$

Since the parameter is updated by gradient ascent, i.e., $\theta_t = \theta_{t-1} + \eta \nabla_{\theta_{t-1}} J_{\theta}(\bar{\mathcal{C}}_t)$, we obtain

$$\begin{aligned} f(\theta_t) &\approx f(\theta_{t-1}) + \nabla_{\theta_{t-1}} f(\theta) \cdot \eta \nabla_{\theta_{t-1}} J_{\theta}(\bar{\mathcal{C}}_t) \\ &= f(\theta_{t-1}) + \nabla_{\theta_{t-1}} f(\theta) \cdot \eta \sum_{x \in \bar{\mathcal{C}}_t} \nabla_{\theta_{t-1}} J_{\theta}(\bar{\mathcal{C}}_t). \end{aligned} \quad (33)$$

Therefore, the subset utility $f(\theta_t)$ becomes the sum of the utilities of the samples.

Regret bound under subset selection. Under the first-order approximation, the subset-level reward is additive. If we treat the advantage-dispatched per-sample reward as the (unbiased) true reward, the resulting problem can be modeled as a contextual semi-bandit. In each round, selecting a size- K subset can be viewed as making K single-arm choices (with distinct arms) and receiving feedback for all selected arms at the end of the round. Therefore, any adversarial single-play regret guarantee extends to the subset setting with at most a factor- K degradation, i.e., the cumulative regret is upper bounded by K times the single-rollout selection bound (up to the same logarithmic factors).

Experiment	M	ϵ_0	η_2	T_w	α	$\epsilon_{\min}$	δ	$p\%$	$e_{\min}$
Qwen3-1.7B-Base									
GRPO + CBS	2048	1	0.0001	50	0.9	0.2	0.008	-	0.1
GRPO + CBS*	1024	1	0.0001	50	0.9	0.2	0.008	30%	0.1
DAPO + CBS	2048	1	0.0001	50	0.9	0.2	0.008	-	0.5
DAPO + CBS*	1024	1	0.0001	50	0.9	0.2	0.008	30%	0.5
GSPO + CBS	2048	1	0.0001	50	0.9	0.005	0.008	-	0.5
GSPO + CBS*	1024	1	0.0001	50	0.9	0.005	0.008	30%	0.5
Qwen3-4B-Base									
GRPO + CBS	2048	1	0.0001	50	0.9	0.2	0.008	-	0.1
GRPO + CBS*	1024	1	0.0001	50	0.9	0.2	0.008	30%	0.1
DAPO + CBS	2048	1	0.0001	50	0.9	0.2	0.008	-	0.5
DAPO + CBS*	1024	1	0.0001	50	0.9	0.2	0.008	30%	0.5
GSPO + CBS	2048	1	0.0001	0	0.9	0.005	0.008	-	0.5
GSPO + CBS*	1024	1	0.0001	50	0.9	0.005	0.008	30%	0.5
Qwen3-8B-Base									
GRPO + CBS	2048	1	0.0001	50	0.9	0.2	0.008	-	0.1
GRPO + CBS*	1024	1	0.0001	50	0.9	0.2	0.008	30%	0.1
DAPO + CBS	2048	1	0.0001	50	0.9	0.2	0.008	-	0.8
DAPO + CBS*	1024	1	0.0001	50	0.9	0.2	0.008	30%	0.5
GSPO + CBS	2048	1	0.0001	50	0.9	0.005	0.008	-	0.5
GSPO + CBS*	1024	1	0.0001	50	0.9	0.005	0.008	30%	0.5

Table 4. Scheduler hyperparameters (adjusted for different models and sizes).

D. Implementation Details

Hyperparameters. We set the learning rate to 1×10^{-6} , the batch size to 128, the mini-batch size to 16, and the rollout number per prompt to 8. For the scheduler, we set the buffer size to 2048 for global scheduling and p to 30% for intra-group scheduling. We keep the mini-batch size unchanged, and the policy optimization step decreases when rollouts are downsampled by the online scheduler. For example, if we generate 1024 rollouts and the scheduler downsamples them to $\lfloor 1024 \times 0.3 \rfloor = 307$, we then rearrange these rollouts into $\lfloor 307 / (16 \times 8) \rfloor = 2$ mini-batches with each having 128 rollouts. The entropy penalty weight w_e is set to 100. In our implementation, we approximate intra-group selection by pooling rollouts from all groups generated at the same step and selecting the top- $p\%$ globally, which yields a comparable retained set while simplifying the scheduler. More method-specific hyperparameters are summarized in Table 4. Following (Liu et al., 2025a), we use the prompt template below,

Prompt Template

```
<|im.start|>system
You are a helpful assistant.<|im.end|>
<|im.start|>user
{Problem}
Please reason step by step, and put your final answer within boxed{ }.<|im.end|>
<|im.start|>assistant
```

Train and evaluation. We control the rollout volume during training to ensure a fair comparison across different policy optimization methods. Specifically, we train GRPO and GSPO for 500 steps. For DAPO, its dynamic sampling requires first upsampling the rollout by a factor of three, followed by downsampling the non-zero advantage ones. Therefore, we train DAPO for 200 steps. During training, we evaluate every 20 steps, reporting Avg@1 for larger datasets (i.e., MATH500, Minerva, and Olympiad) and Avg@16 for smaller datasets (i.e., AIME24 and AIME25). After training, we use the checkpoint with the highest average evaluation score to perform evaluation on the same dataset, computing Avg@4 for larger datasets and Avg@32 for smaller datasets. The temperature and maximum response length are set to 1.0 and 4096, respectively, for both training and evaluation. For the reward function, we use *Math_Verify*⁴ to compare the response with the answer, assigning a value of 1 if they match, and 0 otherwise.

E. More Experimental Results

E.1. Training Dynamics

We show the dynamics of average score on the validation set, training reward and actor update time in Figure 6, Figure 7 and Figure 8 respectively.

E.2. Ablation Study

The dataset-specific performance of different variants is shown in Table 5. Then entropy dynamics of the three PO methods are shown in Figure 9.

E.3. Hyperparameter Sensitivity

To explore the impact of buffer size $|C_t|$, we vary the buffer size of GRPO + CBS under the Qwen3-4B-Base setting, ranging from 1024 to 5120 in increments of 1024, and analyze both the performance and buffer-related runtime. As shown in Figure 11, CBS consistently outperforms the original GRPO within the range of [1024, 4096], demonstrating its robustness. However, when the buffer size reaches 5120, performance degrades below that of GRPO, which is expected due to factors such as increased noise from more data and the fact that the importance sampling in policy optimization is less likely to prioritize outdated data. Besides, the runtime generally increases as the buffer size increase. Moreover, the runtime typically increases with the buffer size. To optimize both performance and efficiency, we set the buffer size to 2048, balancing the tradeoff between performance and runtime.

⁴<https://github.com/huggingface/Math-Verify>

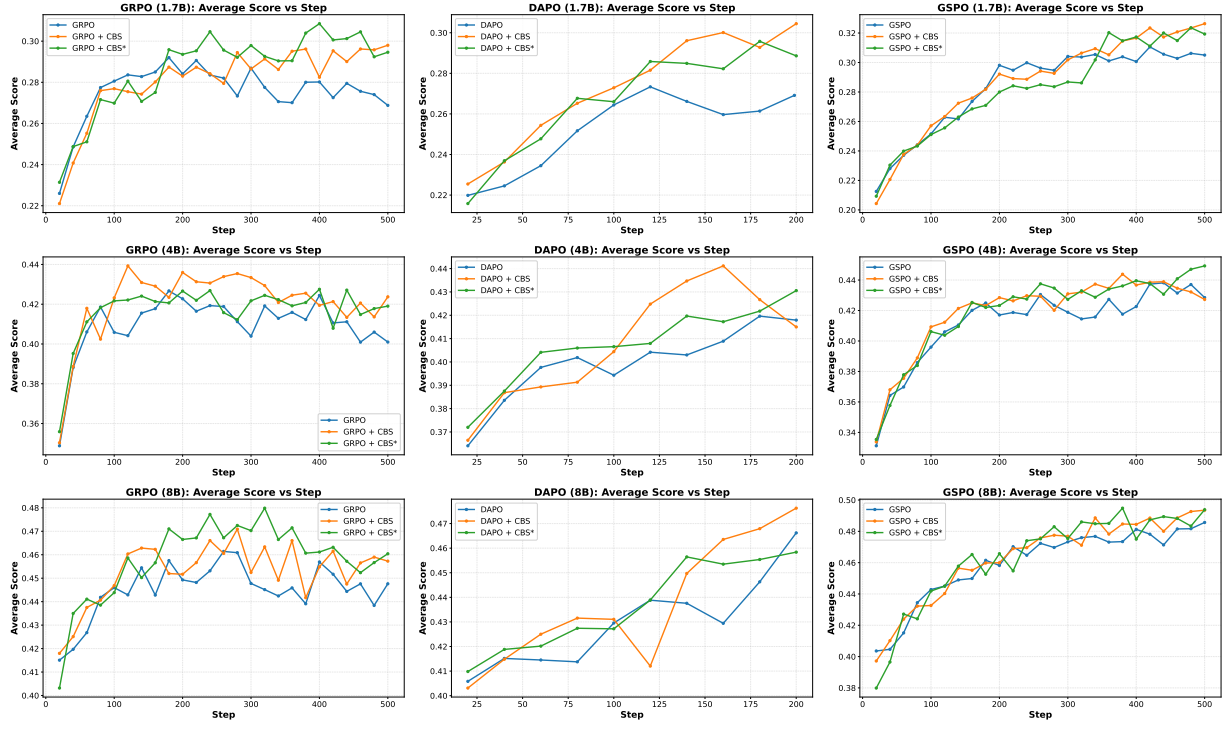


Figure 6. Training dynamics of the average score on the validation set

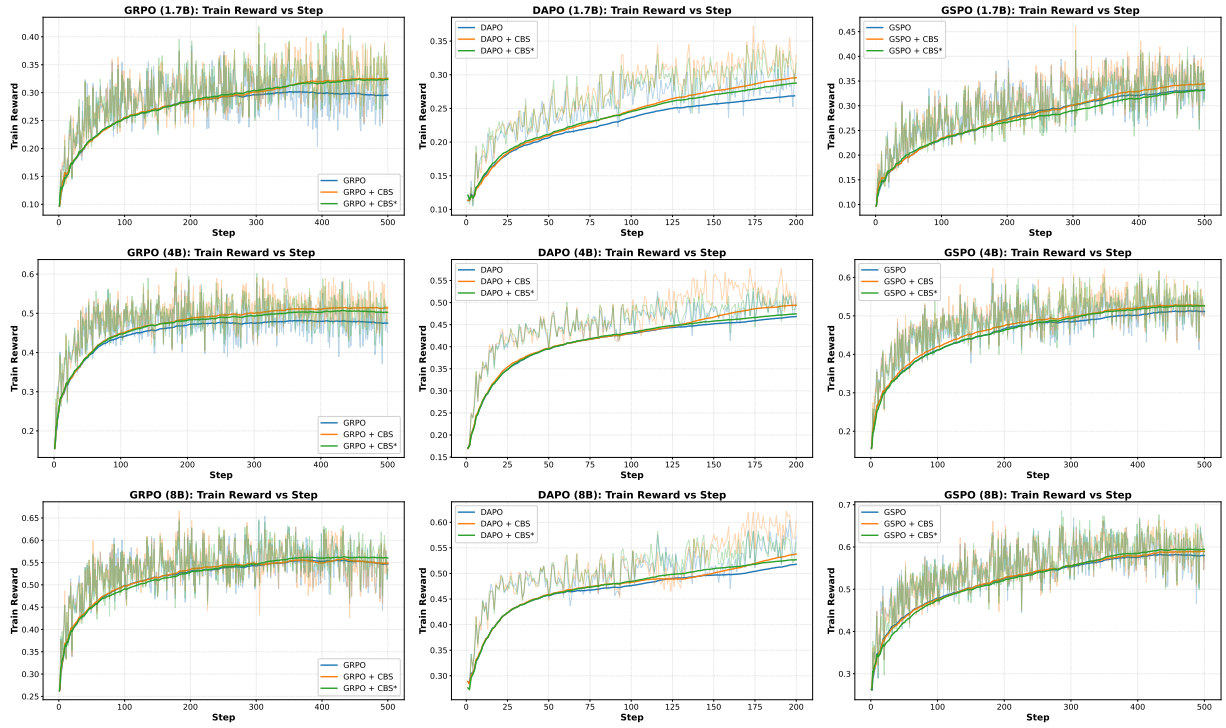


Figure 7. Training dynamics of the train reward.

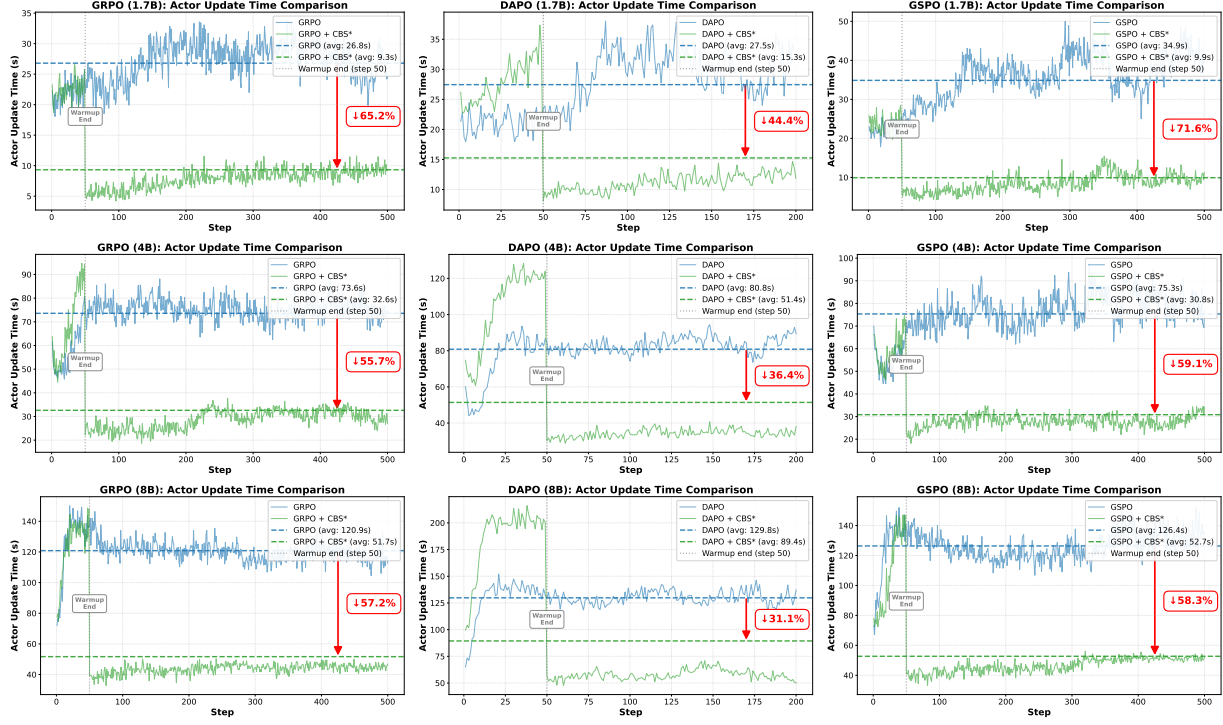


Figure 8. Training dynamics of the actor update time.

Table 5. Dataset-specific ablation study results for Qwen3-4B-Base under global scheduling.

Variant	AIME24	AIME25	AMC23	MATH500	Minerva	Olympiad	Average
<i>GRPO</i>							
CBS	20.31	18.44	63.05	74.45	34.47	43.73	42.41
w/o EMA	20.10	14.37	61.95	74.80	34.65	44.47	41.72
w/o Entropy	19.17	21.15	63.83	74.35	33.00	44.07	42.60
Random	17.19	15.83	61.72	74.85	35.29	43.58	41.41
<i>DAPO</i>							
CBS	20.62	20.94	62.89	75.85	36.58	46.70	43.93
w/o EMA	17.19	16.04	59.30	73.55	34.65	42.21	40.49
w/o Entropy	17.81	17.50	61.64	73.95	31.43	43.32	40.94
Random	15.94	17.08	58.44	73.95	31.07	41.54	39.67
<i>GSPO</i>							
CBS	21.46	18.33	67.03	76.05	36.49	44.18	43.92
w/o EMA	18.02	17.40	64.92	75.50	32.63	44.81	42.21
w/o Entropy	19.17	19.17	67.11	76.65	35.75	44.92	43.80
Random	16.88	15.21	61.17	75.15	32.72	42.14	40.55

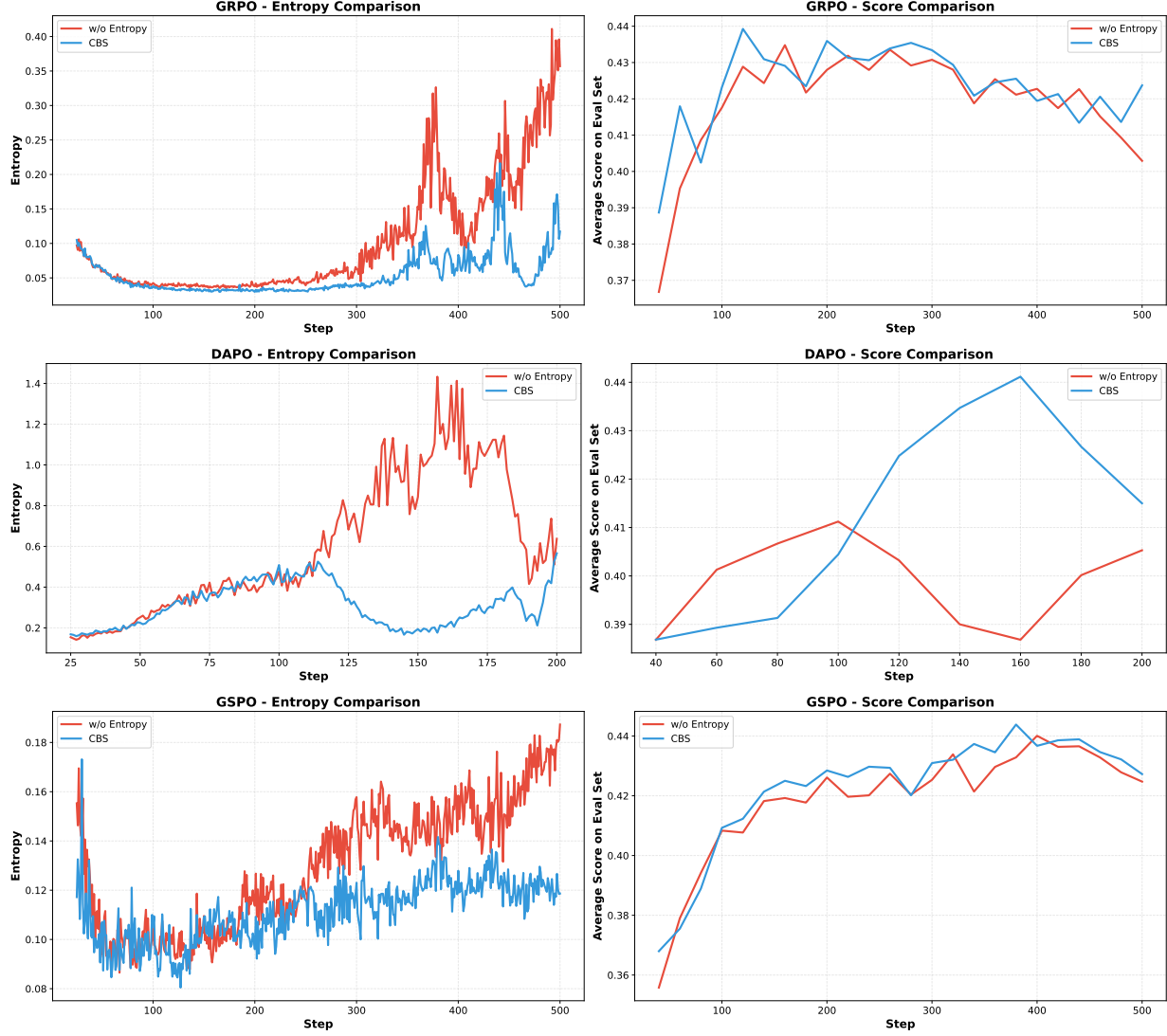


Figure 9. Entropy dynamics of CBS and w/o Entropy.

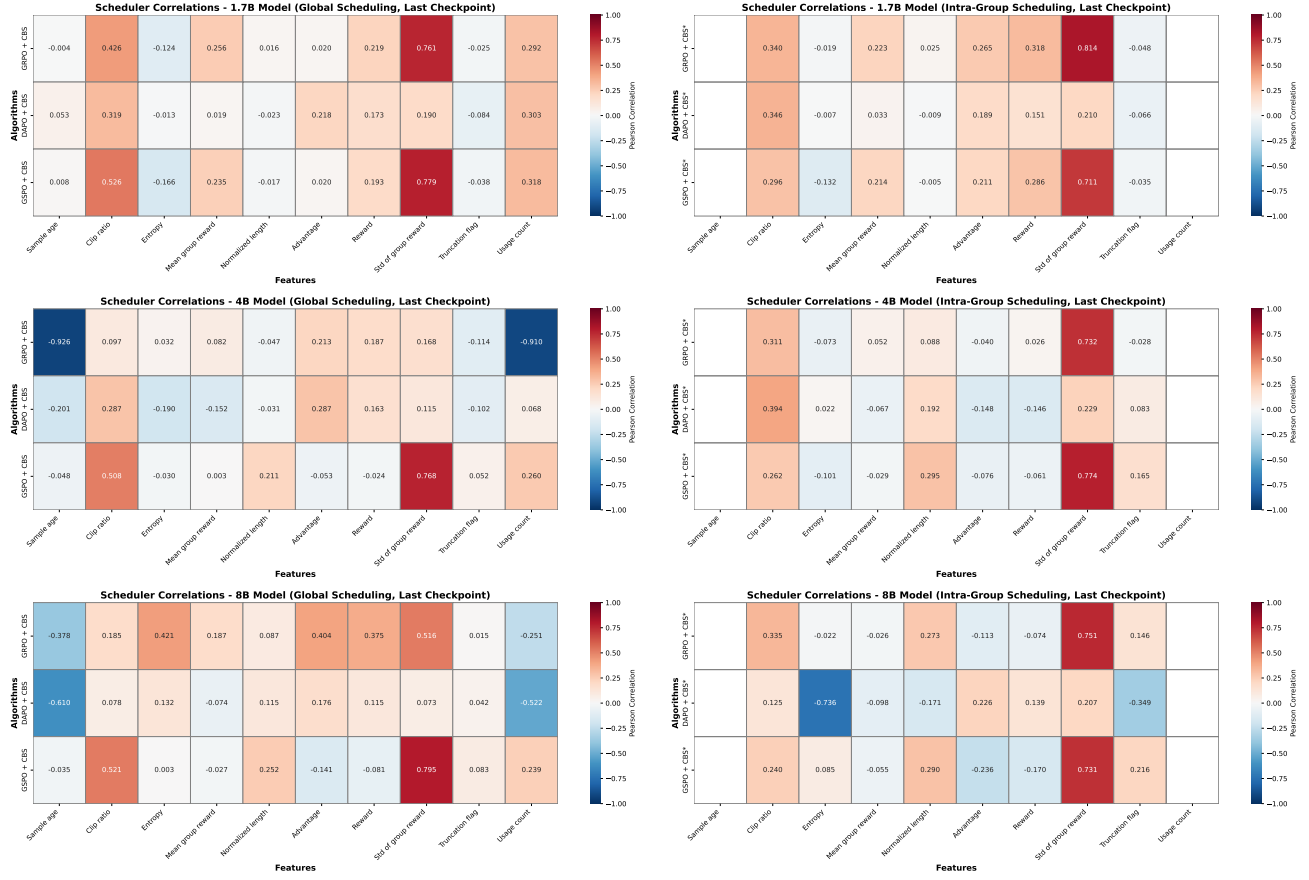


Figure 10. The Pearson correlation coefficient (PCC) between scheduler scores and training dynamic features, with empty blocks representing NaN values due to identical values across the training dynamic features in that dimension.

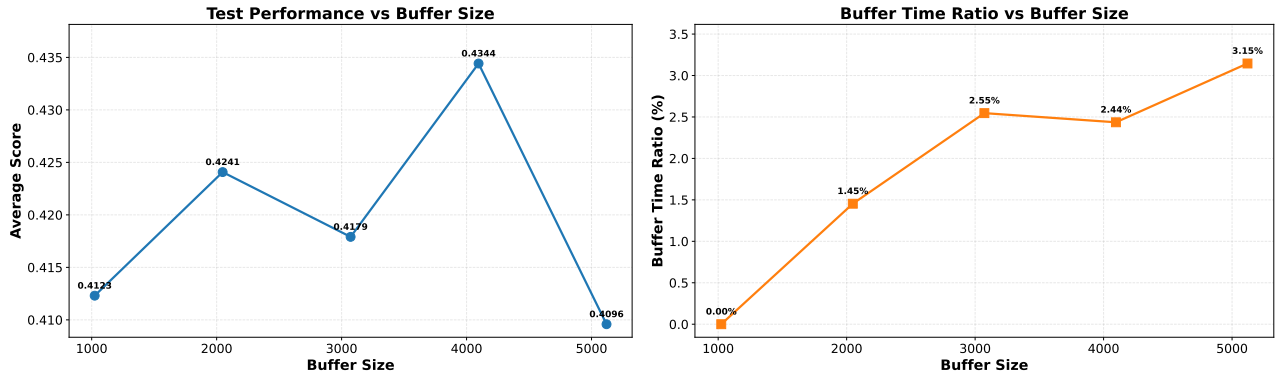


Figure 11. Performance and buffer-related runtime change of buffer size, where buffer size=1024 indicates the original GRPO.

E.4. Scheduling Pattern Analysis and Case Study

We analyze the scheduling pattern of CBS by first examining the Pearson Correlation Coefficient (PCC) between training-dynamics features and the scheduler score. As shown in Figure 10, the patterns differ across policy optimization methods. For instance, GRPO + CBS relies heavily on data interaction history (i.e., usage count and sample age), while GSPO + CBS depends significantly on reward-related signals. These results demonstrate that CBS can adaptively adjust its scheduling behavior based on different policy optimization methods, leading to consistent performance improvements. Furthermore, we also respectively show rollouts that are filtered (in Figure 12) and selected by the scheduler (in Figure 13).

Case Study

Prompt:**system**

You are a helpful assistant.

user

Let n be the largest integer that is the product of exactly 3 distinct prime numbers: x , y , and $10x + y$, where x and y are digits. What is the sum of the digits of n ? Please reason step by step, and put your final answer within `\boxed{\}`.

assistant**Answer:** 12**Model Response:**

Alright, I have this problem in front of me, and I need to find the largest integer n that is the product of exactly three distinct prime numbers: x , y , and $10x + y$, where x and y are single-digit numbers (from 1 to 9). Then, I need to find the sum of the digits of this integer n .

First, let's break it down:

Understanding the Problem

We need three distinct prime numbers: 1. x : a single-digit prime number. 2. y : another single-digit prime number. 3. $10x + y$: a two-digit number that is also prime. And we want the product $n = x \times y \times (10x + y)$ to be as large as possible.

Identifying Single-Digit Prime Numbers

Single-digit primes are: 2, 3, 5, 7. So, both x and y can be any of these values, and they must be different because the primes need to be distinct.

... [Intermediate reasoning steps omitted for brevity] ...

Calculating the Products1. For $x = 2$, $y = 3$:

$$n = 2 \times 3 \times 23 = 6 \times 23 = 138$$

2. For $x = 5$, $y = 3$:

$$n = 5 \times 3 \times 53 = 15 \times 53 = 795$$

Wait, 795 is larger than 138, so it's worth keeping.

3. For $x = 3$, $y = 7$:

$$n = 3 \times 7 \times 37 = 21 \times 37 = 777$$

4. For $x = 7$, $y = 3$:

$$n = 7 \times 3 \times 73 = 21 \times 73 = 1533$$

Now, comparing all the values of n we have: 138, 795, 777, 1533. The largest n is 1533.**Computing the Sum of the Digits of n**

Now, we need to find the sum of the digits of 1533. Let's add them:

$$1 + 5 + 3 + 3 = 12$$

Final AnswerTherefore, the sum of the digits of n is `\boxed{12}`. NullOrUndefinedAssistant systemassistantassistantassistantassistantassistantassistantassistantassistant

... [Output repeats "assistant" continuously until truncate] ...

Figure 12. Rollout filtered by CBS, where the model answers correctly but fails to stop, entering a repetitive loop of token generation.

Case Study

Prompt:**system**

You are a helpful assistant.

user

The positive five-digit integers that use each of the digits 1, 2, 3, 4 and 5 exactly once are ordered from least to greatest. What is the 50th integer in the list?
Please reason step by step, and put your final answer within `\boxed{\}`.

assistant**Answer: 31254****Model Response:**

Alright, I have this problem in front of me:

The positive five-digit integers that use each of the digits 1, 2, 3, 4, and 5 exactly once are ordered from least to greatest. What is the 50th integer in the list?

I need to find the 50th permutation of the digits 1, 2, 3, 4, and 5 arranged in ascending order. Let's break this down.

Understanding the Problem

First, I need to generate five-digit numbers using each digit from 1 to 5 exactly once, sorted from smallest to largest, and then find the number that is in the 50th position.

Total Number of Permutations

I know that the total number of unique five-digit numbers using each of the digits 1, 2, 3, 4, and 5 exactly once is the number of permutations of 5 distinct digits.

The number of permutations of 5 distinct digits is:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

So, there are 120 such numbers, and they are ordered from smallest to largest.

Finding the 50th Permutation

To find the 50th permutation, I think it would be helpful to understand how the permutations are ordered.

Each unique starting digit (first digit) can lead to $4! = 24$ permutations because there are 24 ways to arrange the remaining four digits.

Since $24 \times 2 = 48$, the 50th permutation falls into the range where the first digit is 5 (because the first 48 permutations must have 1, 2, or 3 as the first digit).

Wait, let's check:

- The first 24 numbers start with 1.
- The next 24 numbers start with 2.
- The next 24 numbers start with 3.
- The next 24 numbers start with 4.
- The next 24 numbers start with 5.

So:

- Numbers 1 to 24: first digit = 1
- Numbers 25 to 48: first digit = 2
- Numbers 49 to 72: first digit = 3

Aha, so the 50th permutation is between numbers 49 and 72, which corresponds to the first digit being 3.

Narrowing Down Further

Now, I need to find the 50th permutation. Since the first 48 are covered (24 starting with 1 + 24 starting with 2), the 49th to 72nd permutations start with 3.

Thus, the 50th permutation is the **2nd permutation** in the set of permutations starting with 3 (since $48 + 2 = 50$).

To find permutations starting with 3, the remaining digits are 1, 2, 4, 5. Now, the task reduces to finding the 2nd smallest permutation of the digits 1, 2, 4, 5 with 3 fixed as the first digit.

Finding the 2nd Smallest Permutation of 1, 2, 4, 5

First, find all permutations of 1, 2, 4, 5 ordered from smallest to largest. To find the 2nd permutation, let's list them step by step.

The smallest permutation (1st) will have the smallest possible digits in the subsequent places.

1. **1st permutation:** Start with 1, remaining digits 2, 4, 5 in ascending order: **1 2 4 5**.
2. **2nd permutation:** Keep 1 as the second digit, swap the last two: **1 2 5 4**.

Thus, the second permutation starting with 3 (which is the 50th overall) is **3 1 2 5 4**.

Final Answer

After this reasoning, the 50th five-digit integer using each of the digits 1, 2, 3, 4, and 5 exactly once, in ascending order, is

31254

Figure 13. A rollout elected by CBS, where the answers and CoT are both right.